UNIVERSITÀ DEGLI STUDI DI UDINE
HIC SUNT FUTURA

DEPARTMENT OF MATHEMATICS, COMPUTER SCIENCE AND PHYSICS

BACHELOR THESIS IN
COMPUTER SCIENCE

RAM-USB: Design, implementation and deployment of a multi-user backup service following the zero-trust and defense-in-depth principles

CANDIDATE
Francesco Verrengia

SUPERVISOR
Prof. Ivan Scagnetto

Academic Year 2025-2026

Institute Contacts
Dipartimento di Scienze Matematiche, Informatiche e Fisiche
Università degli Studi di Udine
Via delle Scienze, 206
33100 Udine — Italia
+39 0432 558400
https://www.dmif.uniud.it/

© 2026 Francesco Verrengia
This work is shared under the Creative Commons 4.0 License Attribution-NonCommercial-ShareAlike.

To my best friend Marianna Massari.
The light may flicker,
the light may fade,
even the dark with you feels like the brightest day

Acknowledgements

To my parents, for never doubting me, and for giving me the chance to make mistakes in a thousand ways and learn in a thousand more. Thank you for never trying to influence my choices and for always supporting (and funding) me in them, however crazy they might have seemed.

To my sister Chiara, for never giving up on getting closer to me despite my apathy, and for being my anchor all these years. Know that I love you, even though I always try to run away from your hugs.

To my unstoppable aunt Elvira, who, despite having a thousand things to do, has never missed one of my birthdays or name days, and has always been there, even at the most unexpected moments.

To my cousins, Antonio and Adriana, for trying to get closer to me every time they had the chance, and for asking my opinion on anything even remotely related to computer science.

To my aunt Orsola, who always insists that I come visit her. Thank you for always being there for me and for asking for my advice in the most critical moments.

To my uncle Franco, know that if I have made it this far, it is also thanks to the affection you have always given me. I hope you are proud of who I am today.

To my cousin Alessandro, in the hope that he finds his own path, as I did, without caring about the words of those who cannot understand. Do what you love, cultivate your passions, make mistakes, grow, and try again. Remember that you can always count on me.

To all of you I say thank you, for crossing Italy just to be with me today.

To my friends and fellow students: thank you for sharing with me the best experiences of the last four years.

First and foremost, Ale, for all the walks downtown, the lactose-filled ice creams I should not have eaten, the board game nights, and the long, deep conversations in the Verremobile until two in the morning. For the flashcards, the Pomodoro sessions, and all the study techniques you were so keen to share with me. I still hope you will share all your work with the rest of the community, and that in the future, as a researcher, you will get used to sharing what you learn with the rest of the world.

Ricky, to you more than anyone, I owe a lot. Thank you for backing my idea by taking part in the IoT project, even though it had very little to do with IoT. In any case, thank you, because that is where my interest in Open Source, my passion for design, my work on GitHub, and my contributions to Signal and Lumo were born. All things I am very proud of. Finally, thank you, because this thesis would not exist without you.

Marty, thank you for always being there for me and for keeping me from losing my mind when we were studying Algorithms. Thank you for the evenings spent watching Cydo's livestreams, for introducing me to D&D and making me love it, and for the wonderful experience at Lucca Comics. We literally went through a cyclone and nearly flew off a mountain that evening, but it was an incredible experience that I will never forget. You have a difficult path ahead of you, but I have no doubt that you will make it.

Mattone, I will never forget the day we met, in the computer architecture lab. You deserve a special thank you because without you, this group would never have formed. You are almost there now, you only have 3 exams left, even though they are the toughest in the whole bachelor's program. Do not give up, but please, go a little easier on the "detox". Now that I am a doctor, I can tell you that the phone is bad for your health, especially yours.

Tommy, thank you for the Ferragosto barbecues and for setting an extra place for me at the last minute. I hope your place will remain the group's meeting point for years to come.

Finally, I thank Marianna Massari, without whom I would certainly not be here today. Thank you for sparking in me the passion for computer science, that day when, just so I could play games with you, I built my first PC. Thank you for that December 6, 2022, when you brought light into my darkest moment. Thank you for always being on my side, even when I myself could not be. Thank you for the more than 500 songs you gave me, which I have listened to and loved every day for the last 9 years. Finally, thank you for giving me a name, with which I proudly sign every one of my projects.

Abstract

This thesis presents the design, implementation, and deployment of RAM-USB, a remotely accessible multi-user backup service, designed according to the zero-trust and defense-in-depth principles.

The adopted methodology combines the spiral model for the definition and evolution of the requirements, the Test-Driven Development approach for the implementation, and the V-model for their verification. It also guarantees unidirectional traceability from requirement to code and bidirectional traceability from requirement to commit and from requirement to test.

The work focuses in particular on the design of the authentication and dynamic access control services, as well as on the solutions adopted to protect user credentials, critically analyzing their advantages, disadvantages, and security implications. The system that implements them is an n-tier client-server microservice architecture, organized into components dedicated to access, registration, backup management, and metrics observability, deployed in Docker containers which are in turn isolated in Kernel-based Virtual Machines and Linux Containers on one or more Proxmox hosts.

Internal communications take place via REST APIs for registration and login operations, and via MQTT for the publication of operational metrics; all of them remain confined to the mesh network coordinated by Headscale, mutually authenticated via mTLS with X.509 certificates issued by an internal Certificate-Authority.

In accordance with the zero-trust principle, every component revalidates the format and content of every request, without assuming that the previous component has already done so. Following authentication and authorization, the system dynamically modifies the access rules of the mesh network, allowing the user to reach their own backup space via SFTP, isolated from that of the other users by means of chroot. Backup files are deduplicated, compressed, and encrypted by Restic before leaving the user's device, and their confidentiality therefore remains guaranteed by client-side encryption.

The thesis concludes with a critical assessment of the security, confidentiality, and functionality properties of the system in the scenario of full compromise of a single service. The analysis makes it possible to assess the effectiveness of the isolation measures and of the defense-in-depth strategies adopted, while highlighting their limitations and the properties that would be preserved even in the presence of a severe compromise of the service.

Contents

List of Tables xiii

List of Figures xv

List of acronyms xix

1 Introduction 1
- 1.1 Background . . . 1
  - 1.1.1 Hash functions . . . 2
- 1.2 Problem statement . . . 2
- 1.3 Objectives . . . 2
- 1.4 Methodology . . . 2
- 1.5 Thesis structure . . . 3

2 Foundational principles 5
- 2.1 Zero-trust . . . 5
- 2.2 Defense-in-depth . . . 5

3 Requirements: methodology and use cases 7
- 3.1 Requirements methodology . . . 7
- 3.2 Requirements traceability . . . 7
- 3.3 User requirements and use cases . . . 8

4 Overall architecture and trust zones 11
- 4.1 Architecture overview . . . 11
  - 4.1.1 Entry-Hub . . . 11
  - 4.1.2 Security-Switch . . . 12
  - 4.1.3 Database-Vault . . . 12
  - 4.1.4 Storage-Service . . . 13
  - 4.1.5 Network-Manager . . . 13
  - 4.1.6 Certificate-Authority . . . 13
  - 4.1.7 Headscale . . . 14
  - 4.1.8 Mosquitto . . . 14
  - 4.1.9 Metrics-Collector . . . 14
  - 4.1.10 Grafana . . . 14
- 4.2 Inter-service communication model . . . 15
- 4.3 Trust zones and security boundaries . . . 16
- 4.4 PKI and certificate management . . . 17

5 Authentication and identity management 19
- 5.1 Request lifecycle . . . 19
- 5.2 Credential encryption and hashing . . . 20
  - 5.2.1 Password strength . . . 22
- 5.3 Registration . . . 24

5.4 Login . . . 25
5.5 Post-authentication orchestration . . . 27

6 File storage and backup **29**
6.1 SFTP-only access model . . . 29
6.2 Isolation and dynamic chroot . . . 30
6.3 Backup and restore . . . 30

7 Networking and mesh coordination **33**
7.1 Network policies and reachability boundaries . . . 33
7.2 Access grant lifecycle . . . 33
7.3 The coordinator paradox: Headscale as a standalone service . . . 35
7.4 CG-NAT and the need for a dedicated public endpoint . . . 36

8 Observability and metrics **37**
8.1 Publishing the metrics . . . 37
8.2 Reception and consistency check . . . 38
8.3 Time-series persistence . . . 38
8.4 Metrics visualization . . . 38

9 Testing and validation **41**
9.1 The V-model and the four test levels . . . 41
9.2 Bottom-up integration strategy . . . 41
9.3 TDD: entry and exit criteria . . . 42
9.4 Coverage and gaps in the test plan . . . 42
9.5 Automated pre-commit checks . . . 43
9.6 Results: completed verifications . . . 43

10 Deployment **45**
10.1 Containerization and s6-overlay supervision . . . 45
10.2 Why not all containers are distroless . . . 45
10.3 Local Docker vs Proxmox VE: two levels of isolation . . . 46
10.4 Distributed topology . . . 47
10.5 High availability and resilience: current state and limitations . . . 48

11 Critical analysis of security properties **49**
11.1 Attack scenarios and mitigations . . . 49
11.1.1 Email address enumeration at registration . . . 49
11.1.2 Administrative access to credentials . . . 50
11.1.3 Offline address verification from a database copy . . . 50
11.2 Failure isolation . . . 51
11.2.1 Entry-Hub . . . 51
11.2.2 Certificate-Authority . . . 52
11.2.3 Database-Vault . . . 52
11.2.4 Network-Manager . . . 53
11.2.5 Storage-Service . . . 53

12 Conclusions **55**
12.1 Achieved results . . . 55
12.2 Known limitations . . . 56
12.3 Future work . . . 56

A Complete list of requirements **57**
A.1 Functional system requirements . . . 57
A.1.1 Client . . . 57
A.1.2 Entry-Hub . . . 58
A.1.3 Security-Switch . . . 59
A.1.4 Database-Vault . . . 59
A.1.5 Storage-Service . . . 61
A.1.6 Network-Manager . . . 62
A.1.7 Certificate-Authority . . . 63
A.1.8 Monitoring system . . . 63
A.1.9 Network infrastructure and PKI . . . 64
A.2 Non-functional requirements . . . 64
A.2.1 Product . . . 64
A.2.2 Organizational . . . 65
A.2.3 External . . . 65
A.3 Domain requirements . . . 65

Bibliography **67**

List of Tables

5.1 HTTP status codes returned to the client and their meaning in the system. 20
5.2 Fields of the PHC string in which Database-Vault stores the password hash. 22

9.1 Verification level associated with each specification level. 41
9.2 Properties verified on the running system. 44

10.1 Base image of each service and the processes sharing its container. 46

11.1 User Requirements verified in this section against the full compromise of a service. . . . 51

A.1 Functional requirements of the client. 57
A.2 Functional requirements of Entry-Hub. 58
A.3 Functional requirements of Security-Switch. 59
A.4 Functional requirements of Database-Vault. 60
A.5 Functional requirements of Storage-Service. 61
A.6 Functional requirements of Network-Manager. 62
A.7 Functional requirements of Certificate-Authority. 63
A.8 Functional requirements of the monitoring system (MQTT-Broker / Metrics-Collector / TimescaleDB / Grafana). 63
A.9 Functional requirements of network and PKI. 64
A.10 Product non-functional requirements. 64
A.11 Organizational non-functional requirements. 65
A.12 External non-functional requirements. 65
A.13 Domain requirements (RD). 65

List of Figures

3.1 Use case diagram of RAM-USB's main use cases. 9

4.1 Entity-Relationship diagram of the user record in Database-Vault. 12
4.2 Storage-Service filesystem structure, from `creator.go` and `dirmaker.go` [1, 2]. 13
4.3 Container diagram of RAM-USB. 15
4.4 Trust zones and reachability on the mesh network. 16
4.5 Hierarchy of the certificate authorities and signing relationships. 17

5.1 Registration sequence (UC-01). 25
5.2 Authentication sequence (UC-02). 26

6.1 Sequence for backing up a file (UC-03). 31

7.1 Lifecycle of the access grant associated with a user's node. 35

8.1 Observability pipeline, from the publication of the metrics to the Grafana dashboard. . 39

10.1 Target production topology. 47

List of acronyms

Acronym	Meaning
ACL	Access Control List
AES-GCM	Advanced Encryption Standard, Galois/Counter Mode
API	Application Programming Interface
CA	Certificate Authority
CG-NAT	Carrier-Grade Network Address Translation
CLI	Command Line Interface
DNS	Domain Name System
GDPR	General Data Protection Regulation
HKDF	HMAC-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
HTTP/HTTPS	HyperText Transfer Protocol (Secure)
IP	Internet Protocol
JSON	JavaScript Object Notation
KVM	Kernel-based Virtual Machine
LXC	Linux Container
MQTT	Message Queuing Telemetry Transport
mTLS	mutual Transport Layer Security
NAT	Network Address Translation
NIST	National Institute of Standards and Technology
OWASP	Open Web Application Security Project
PDF	Portable Document Format
PHC	Password Hashing Competition
PKI	Public Key Infrastructure
POSIX	Portable Operating System Interface
QoS	Quality of Service
RAID	Redundant Array of Inexpensive Disks
REST	REpresentational State Transfer
RFC	Request for Comments
SFTP	Secure Shell File Transfer Protocol
SHA	Secure Hash Algorithm
SRS	Software Requirements Specification
SSH	Secure Shell

Acronym	**Meaning**
TCP	Transmission Control Protocol
TDD	Test-Driven Development
TLS	Transport Layer Security
UID	User Identifier
VE	Virtual Environment
VPS	Virtual Private Server

1

Introduction

1.1 Background

A backup, in the field of cybersecurity, is a disaster recovery process: a procedure that makes it possible to recover lost data, for example following an attack, a hardware or software failure, or a human error. One of the first technical responses to data loss is the redundancy of the disks on which the data is stored: in 1988 Patterson, Gibson and Katz formalized RAID (Redundant Array of Inexpensive Disks) [3], which distributes the same data across multiple inexpensive disks to obtain a reliability comparable to that of a single, more expensive disk.

This redundancy, however, remains local and does not protect against an event that destroys the entire installation. From this arises the need for a remote backup: a copy of the data kept in a place other than that of the originals. The simplest way is to entrust it to third parties, an acquaintance who owns the hardware or a business that offers this kind of service, but this requires trusting that the data will not be read, modified, or sold by whoever keeps it.

Whoever offers the service, for their part, finds it convenient to serve multiple users at the same time. This, however, requires distinguishing between users, and guaranteeing that each one accesses only their own data: in practice, it requires an authentication and access control system.

Among the various existing authentication systems, the one adopted and discussed in this thesis is based on email and password which, being sensitive data, require a high level of protection. The principle on which this kind of authentication rests is simple: the user registers by sending their credentials to the server; during login they send them again, and the server compares them with those saved at registration. If they match, the user is authenticated.

The fact that the server holds the credentials, however, exposes all users to the risk of their being stolen. Numerous systems exist to prevent this; one of these, examined in depth in this thesis, is the use of a hash function. The server, instead of saving the credentials in plaintext, saves them in the form of a hash. During login, it recomputes the hashes of the credentials provided by the user and checks whether they match the saved ones.

1.1.1 Hash functions

Hash functions are non-invertible functions that transform an input of arbitrary length into an output of fixed length, strictly correlated with the input data, called a hash. Denoting a hash function by H, it enjoys the following fundamental properties:

- **preimage resistance:** given h, it is computationally intractable to find x such that $H(x) = h$;
- **second-preimage resistance:** given x, it is computationally intractable to find $y \neq x$ such that $H(x) = H(y)$;
- **collision resistance:** it is computationally intractable to find any pair (x, y) with $x \neq y$ such that $H(x) = H(y)$.

1.2 Problem statement

Among the various causes of data loss, the project focuses on the one arising from an attack by a malicious actor, adopting the principles of zero-trust and defense-in-depth to guarantee the security of the data even if any component of the system is compromised.

1.3 Objectives

RAM-USB, Remotely Accessible Multi-User Secure Backup, is an in-depth case study on the design, implementation and deployment of a backup service that satisfies the requirements described in Chapter 3. It is not meant to compete with existing commercial solutions, but to discuss its own design choices, illustrating their pros and cons transparently, with a security-first approach focused on defending the system against external attacks.

1.4 Methodology

Some of the components of RAM-USB originate from the brainstorming phase carried out together with Riccardo Gottardi [4].

The adopted methodology combines the spiral model for the definition and evolution of the requirements [5] with the Test-Driven Development approach for the implementation and the V-model for their verification [6].

The language used for the entire stack is Go, a compiled language explicitly designed to handle many simultaneous network connections efficiently.

Bash was used to orchestrate the infrastructure around the Go code (starting the services, supervising the processes in the containers, sequencing at startup).

The code is versioned with Git and published free of charge, in its entirety, on GitHub, distributed under the MIT license.

The implementation was carried out entirely with the assistance of the Large Language Models Claude Sonnet 4.5 and Sonnet 5, in multi-agent form through the Claude Code harness running in a virtual machine on Proxmox, for security and privacy reasons.

Generative artificial intelligence was employed exclusively as a tool under close human supervision. The design, the structure of the code, the module boundaries, the signatures, the behavior of the individual functions, and the corresponding tests remain a human decision; the assistant implemented only what was specified, in small, incremental, and verifiable steps, exercising latitude only over low-level implementation details. Every change was reviewed and explicitly approved before being introduced into the codebase. The numerous strategies for mitigating implementation errors and hallucinations of the two models, as well as the management of the multiple agents used, are not the subject of this thesis.

1.5 Thesis structure

Chapter 2 examines in depth the two security principles on which RAM-USB is built, zero-trust and defense-in-depth.

Chapter 3 describes the methodology by which the requirements were gathered and traced, and the use cases that derive from them.

Chapter 4 presents the components that take part in registration and login.

Chapter 5 follows the credentials along that path, from the public boundary to the saved record, and describes the rules that each layer applies.

Chapter 6 follows the backup to its own isolated space on Storage-Service.

Chapter 7 describes how the mesh network enforces the reachability boundaries between the components.

Chapter 8 closes the lifecycle of a request with the observability pipeline that makes it measurable.

Chapter 9 verifies the system thus built through the four levels of the V-model, from unit testing to the acceptance checklist, and reports what running it on real hardware revealed.

Chapter 10 describes how these components are actually deployed, across Docker containers, Proxmox machines, and a public VPS.

Chapter 11 rereads the entire system from the point of view of whoever attempts to compromise it, verifying which guarantees hold even against its full compromise.

Chapter 12 closes the thesis by weighing the results obtained against the limitations encountered, and indicating the most relevant future work.

2

Foundational principles

2.1 Zero-trust

The term “zero trust” was coined in 2010 by Forrester analyst John Kindervag. The underlying idea was not to grant trust on the basis of network location, but to verify every access independently of it.

NIST Special Publication 800-207 [7] today formalizes this principle as the assumption that no network, internal or external, should be considered implicitly trustworthy, and that every access request should be evaluated independently of its origin.

RAM-USB extends the zero-trust principle by distrusting even the identity of a component authenticated via mTLS. This is because every component assumes that the others are under the control of an attacker.

2.2 Defense-in-depth

NIST Special Publication 800-53 Revision 5 defines defense-in-depth as a strategy that integrates people, technology, and operational capabilities to establish redundant barriers across multiple layers of an organization, so that the failure or circumvention of a single barrier does not compromise the system as a whole [8].

3

Requirements: methodology and use cases

3.1 Requirements methodology

The requirements of RAM-USB are collected in a single document called the Software Requirements Specification (SRS) [9], which constitutes the single source of truth about what the system must do. The writing of this document follows a spiral model of software engineering [5], driven by the progressive resolution of risk, understood as the most significant sources of uncertainty in the project. The level of detail of the SRS consequently increases at every iteration because, although the objectives and the high-level requirements were clear from the initial phases, the knowledge of the third-party tools and technologies was not sufficient to predict with certainty the low-level behavior of these tools. The choice of the spiral model stems from this imbalance. Indeed, many concrete constraints emerged only at implementation time, when direct interaction with these tools revealed details that a purely theoretical analysis phase was not able to anticipate. A concrete case of this dynamic is requirement `NM-F-12`, revisited in three successive rounds:

in version 1.0 of the SRS [10] the requirement assumed that Network-Manager (the component responsible for the private network of the system) had to build from scratch its own mechanism for authorizing the access of new devices;

in version 1.4 [11] it emerged that Headscale, the network coordination tool employed by the system, already offered this functionality through its own CLI, eliminating the need to write it;

in version 1.7 [12] a deeper limitation of Headscale itself emerged (it cannot coordinate a network it is part of), examined in depth in Section 7.3, requiring a broader reformulation of the requirement.

3.2 Requirements traceability

A distinctive methodological aspect of the project is traceability, understood as a technical constraint verifiable in three independent directions. This approach favors ease of security auditing at the cost of a more complex implementation workflow.

- **From the requirement to its implementation**: every requirement of the SRS that already

has an implementation is linked to a GitHub permalink pointing to a specific commit and to a precise range of lines.

- **From the test to the requirement and vice versa**: every test function carries, as a documentation comment on the line immediately preceding it, the identifier of the requirement it verifies. The relation is not necessarily one-to-one, since a requirement can have multiple tests that verify it. Moreover, traceability holds in both directions: one can go from the requirement to the tests, by searching for the ID in the codebase; and from the test to the requirement, by reading the comment above the function.
- **From the commit to the requirement**: every commit in the repository follows a fixed grammar, described in `CONTRIBUTING.md` [13] and summarized here as type, area, description of the change, followed by the list of identifiers of the requirements it covers. As with the test-requirement link, this too holds in both directions: from the requirement, one searches for its ID in the repository history; in the opposite direction, the final list in the description of each commit indicates which requirements are covered.

3.3 User requirements and use cases

The user requirements (RU) express, from the point of view of those who use the system, what RAM-USB must guarantee. They are the only requirements reported in full in the body of the thesis; the system requirements that realize them (functional, non-functional, and domain) are too numerous for a complete list, and are collected in Appendix A.

RU-01 Register for the service by providing the minimum amount of data necessary.

RU-02 Be able to authenticate in sessions following registration.

RU-03 Be able to upload encrypted backups to the service from anywhere.

RU-04 Be able to access one's own backup only after having authenticated.

RU-05 Have the guarantee that no one, administrators included, can read one's own backups.

RU-06 Have the guarantee that no one, administrators included, can read one's own access credentials.

RU-07 Be able to retrieve one's own files at any time.

RU-08 Have the certainty that one's own data is isolated from that of other users.

RU-09 *(Out of scope)* Have the guarantee that no one, administrators included, can modify or delete one's own backups.

RU-10 (Administrator) Be able to observe the health and performance of the system without compromising user data.

The user requirements translate into the following use cases, also illustrated in Figure 3.1.

UC-01 (Registration) A new user provides an email, a password, and an SSH public key, and obtains an active account.

UC-02 (Authentication) A registered user provides email and password, and obtains temporary access to their own backup space.

UC-03 (Backing up a file) An authenticated user uploads a file, already encrypted on their own device, to their own isolated space.

UC-04 (Restoring a file) An authenticated user retrieves one of their own previously uploaded files, decrypting it locally.

UC-05 (Consulting operational metrics) An administrator consults the health status and performance of the system, without accessing the data of individual users.

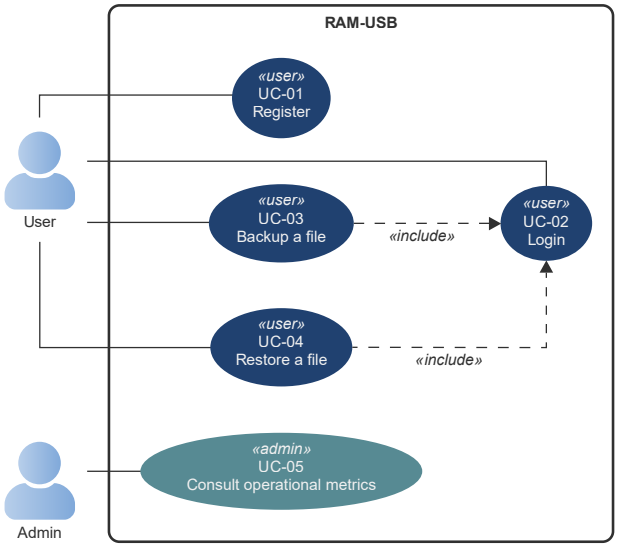


Figure 3.1: Use case diagram of RAM-USB's main use cases.

The following chapters describe how the architecture of the system realizes these use cases: Chapter 4 introduces the overall structure of the components and the security boundaries between them, while Chapters 5–8 examine each functional domain in detail.

4

Overall architecture and trust zones

4.1 Architecture overview

RAM-USB is an n-tier microservice architecture in which each component has a specific responsibility. This separation reduces the risk that the compromise of one component spreads to the entire system.

The architecture of RAM-USB consists of ten components, organized into four functional groups:

- **Access and registration**: Entry-Hub, Security-Switch, Database-Vault.
- **Backup and restore**: Storage-Service.
- **Observability**: Mosquitto, Metrics-Collector, Grafana.
- **Network and certificates**: Network-Manager, Headscale, Certificate-Authority.

4.1.1 Entry-Hub

Entry-Hub is the public gateway of the system, that is, the component exposed to anyone on the internet. For this reason the system grants it no implicit trust. Its process therefore runs in a distroless container [14], following the principle of attack surface reduction recommended by NIST SP 800-190 [15]. Indeed, without a shell, most of the known critical vulnerabilities in containers cannot be exploited [16].

Entry-Hub exposes three HTTPS endpoints: `GET /api/health` and `POST /api/users` are reachable by anyone on the internet, with a certificate signed by the public CA Let's Encrypt [17]. `POST /api/login` uses the same certificate, but has a separate listener, reachable only by users with access to the private network, described in detail in Chapter 7.

When it receives a request, Entry-Hub validates the JSON body in two phases: during decoding, it enforces a size limit and rejects any unexpected field in the payload; then it validates the individual email and password fields and, only during registration, the SSH key [18]. If any of these checks fails, it responds with HTTP 400 without specifying what the problem is and without forwarding the request to the inner layers of the system. If validation succeeds, it forwards the request to Security-Switch over mutual TLS (mTLS), verifying that the received certificate really belongs to a Security-Switch. It waits for the Security-Switch's response and relays it to the client.

4.1.2 Security-Switch

Security-Switch exists to keep the responsibility for contacting Database-Vault and Network-Manager out of the hands of the system's entry point: it receives incoming requests only from Entry-Hub, authenticated via mTLS, revalidates them independently, and contacts the internal services. The validation, error handling, and logging logic is reused by all the main services and lives in shared internal libraries [19].

The purpose of the revalidation is to apply the defense-in-depth principle [8].

If validation succeeds, it forwards the request to Database-Vault via mTLS.

In the case of a registration request, it asks Network-Manager to create the new user's identity on the mesh network and the credential (pre-auth key) with which the user's device will be able to join it.

In the case of a login request, it asks Network-Manager to grant that user twelve-hour temporary access to Storage-Service. Only once the user has obtained access to Storage-Service will they be able to contact it and manage their own backups.

4.1.3 Database-Vault

Database-Vault is the component responsible for storing user credentials in a PostgreSQL database. It is the only one able to access the database, but it nevertheless has no access to user files, which are persisted on Storage-Service. This separation limits the impact of the compromise of a single component.

The Entity-Relationship diagram is shown in Figure 4.1. Since the database must contain only user data and there is no meaningful relationship to store, the ER diagram is rather simple.

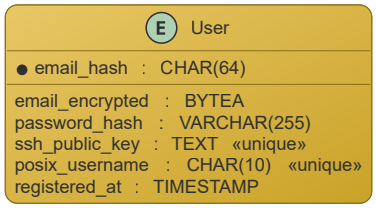


Figure 4.1: Entity-Relationship diagram of the user record in Database-Vault.

The email is encrypted, recoverable only with the master key, while the password is never recoverable, since it is stored only in hashed form, as an Argon2id digest. Theft of, or unauthorized access to, the database is therefore not enough to read either of them.

When a new user registers, after revalidating the input, Database-Vault attempts to save the data in an atomic transaction, rejecting the request if the email or the SSH key is already registered, but without specifying which of the two is wrong. If the save succeeds, it asks Storage-Service to create the user's POSIX space. If Storage-Service fails, it deletes the record just created; otherwise it informs Security-Switch that the registration succeeded.

At login, it recomputes the hash of the received password and compares it with the stored one, then responds to Security-Switch.

A nonexistent email and a wrong password produce the same response: in this way, whoever attempts to log in cannot use it to discover which emails are registered. Chapter 11, however, explains how an external user can try to determine whether an email is already registered in the system, and also how a malicious administrator can obtain a user's access credentials.

4.1.4 Storage-Service

Storage-Service hosts the users' backup files. It never processes content in plaintext because, by design, it is not able to. Indeed, it receives only files that have been deduplicated, compressed, and encrypted client-side with Restic [20].

At Database-Vault's request, it creates a dedicated POSIX user, with neither an interactive shell nor a traditional home directory. Each user instead has a writable space in their own subdirectory inside a chroot. The filesystem structure is shown in Figure 4.2.

```
/storage/ ..... root of all registered users
└─ user<xxxxxx>/ ..... owner root:root, permissions 0711
   └─ data/ ..... owner user<xxxxxx>, permissions 0700
```

Figure 4.2: Storage-Service filesystem structure, from `creator.go` and `dirmaker.go` [1, 2].

Access to Storage-Service can only take place via the Secure Shell File Transfer Protocol (SFTP) [21], with the SSH public key sent by the user during registration. Therefore neither username and password authentication nor any other form of Secure Shell access outside SFTP is permitted.

On every connection, a dedicated `AuthorizedKeysCommand` binary queries Database-Vault to obtain the user's current public key. Any error in that call denies the connection, according to the fail-secure principle [22].

4.1.5 Network-Manager

Network-Manager is the administrator of the network rules. It is responsible for defining, modifying, and deleting the rules that allow a user of the private mesh network to reach the other members.

During registration, it creates a dedicated Headscale user and the pre-auth key with which the user's device will join the network.

During login, it assigns the user's node the Access Control List (ACL) tag that enables twelve-hour temporary access to Storage-Service. A periodic process checks expiries and removes the tag from expired nodes.

4.1.6 Certificate-Authority

Certificate-Authority is the system's private CA, based on `smallstep/certificates` [23].

It issues, renews, and revokes the mTLS certificates of every internal service, which makes it a particularly critical component of the system, as well as the only one reachable by all internal services: a malicious actor who managed to compromise it could issue forged certificates for any organization.

Each service obtains its first certificate by contacting the CA using a bootstrap token distributed manually by an operator out-of-band. A detailed explanation can be found in Section 4.4.

4.1.7 Headscale

Headscale is a self-hosted implementation of the Tailscale control server [24].

It acts as the coordinator of the private mesh network: on every connection attempt between two nodes, it enforces the reachability rules decided by Network-Manager. It also distributes, through MagicDNS, the hostnames with which nodes resolve one another, allowing the client to locate Storage-Service without knowing its IP address.

It also validates the pre-auth key of a new device at the moment it joins the mesh.

Unlike every other internal component except Entry-Hub, Headscale does not reside in the private network but is reachable by anyone on the internet. Section 7.3 explores the reasons in depth.

4.1.8 Mosquitto

Mosquitto is the MQTT broker to which every service, once a minute, publishes its own operational metrics.

It enforces an ACL under which each service can write to its own topic, without ever being able to read from it, while Metrics-Collector can only read the entire `metrics/#` space, without ever writing to it.

4.1.9 Metrics-Collector

Metrics-Collector is the component that receives, validates, and persists the operational metrics published by every service.

It discards every received metric whose `service` field does not match the topic it arrives from, and persists the accepted metrics as a hypertable in TimescaleDB [25], with a retention of thirty days and compression after seven days.

4.1.10 Grafana

Grafana is the visualization platform with which an administrator consults the health status and the performance of the system [26].

It reads from TimescaleDB directly, without going through Metrics-Collector. The Grafana dashboard shows response times, throughput, and active connections. Chapter 8 covers this observability pipeline in detail.

Figure 4.3 summarizes the architecture just described together with the communication protocols used.

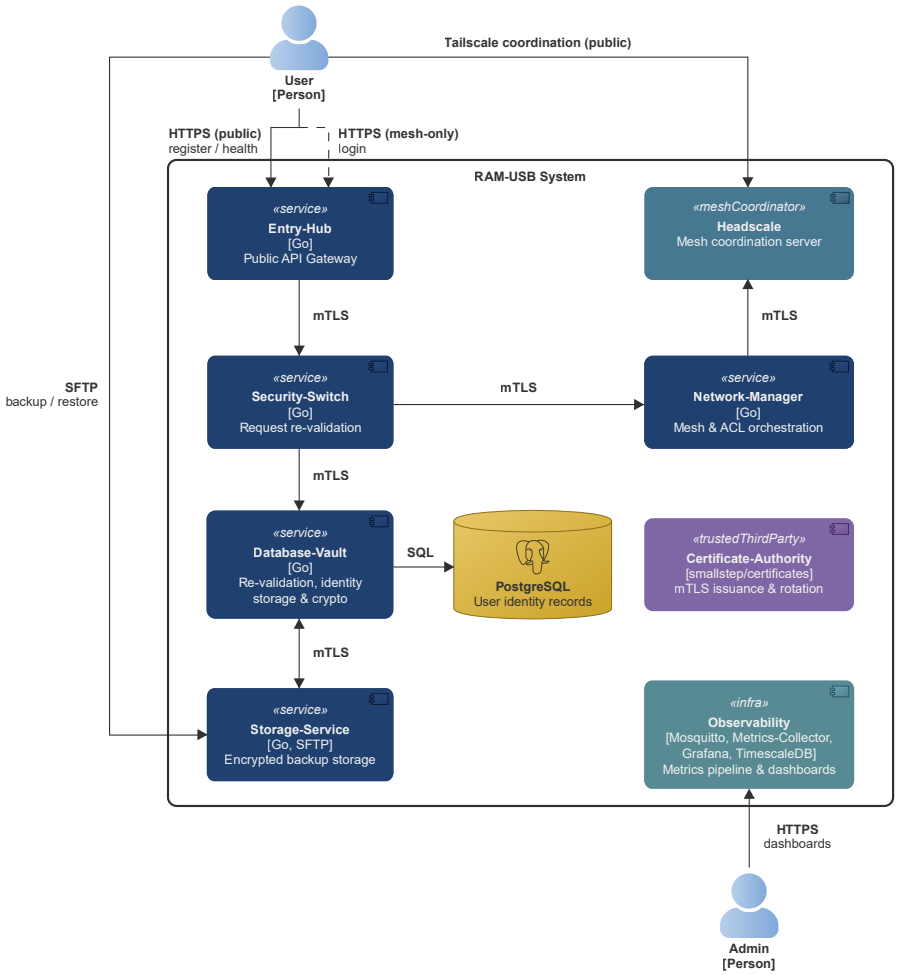


Figure 4.3: Container diagram of RAM-USB.

4.2 Inter-service communication model

Each service exposes its own HTTPS API built on Go's standard library, `net/http`. Requests and responses are JSON bodies, and each endpoint is registered on a minimal router, `http.ServeMux`. The minimum allowed TLS version is 1.3.

The endpoints listed here follow REST conventions [27].

- `GET /api/health`, `POST /api/users` on Entry-Hub,
- `GET /internal/v1/public-key/{posix_username}` on Database-Vault,
- `POST /internal/v1/posix-users` on Storage-Service, and
- `POST /internal/v1/mesh-users`, `POST /internal/v1/grants` on Network-Manager

`POST /api/login` is the only exception, because a successful login returns only the confirmation that an action took place, with no identifier that a real `sessions` resource would require in order to be read with GET or deleted with DELETE. The twelve-hour ACL token that the login grants remains internal to Network-Manager and expires on its own.

The publication of metrics, moreover, is asynchronous: each service publishes its own messages on a dedicated Message Queuing Telemetry Transport (MQTT) topic without any need to wait for a response.

This system structure comes at a significant cost: every registration and login passes through four distinct processes, each with its own mTLS handshake and its own revalidation. Latency is therefore much higher than in a monolithic system, and, should it ever need to be reduced, it cannot be reduced by skipping one of the steps. Moreover, the service remains available to users only as long as the seven components that take part in a request's path remain available, while the observability chain can go down without the user noticing.

The two costs, however, are not of the same kind. Latency is inherent to the design and accepted together with the architecture; availability, by contrast, is a deployment problem, and Chapter 10 discusses to what extent the underlying infrastructure is able to mitigate it.

4.3 Trust zones and security boundaries

Figure 4.4 shows the reachability rules that Network-Manager applies to the mesh network.

The two outermost zones belong to the user, before and after authentication. A user who is registered but not yet authenticated, and who therefore belongs to the *Unauthenticated User Zone*, sees and can contact only Entry-Hub. After login, the same node enters the *Authenticated User Zone* and obtains temporary reachability to Storage-Service.

Certificate-Authority reaches only the MQTT broker, but is reached by every other internal component, to allow it to renew its own X.509 certificate.

The fourth zone, the *Public Zone*, contains only Headscale, reachable from the public network. Section 7.3 explains the reason in detail. However, Network-Manager's calls to Headscale are still protected by mTLS authentication and by the Headscale API key.

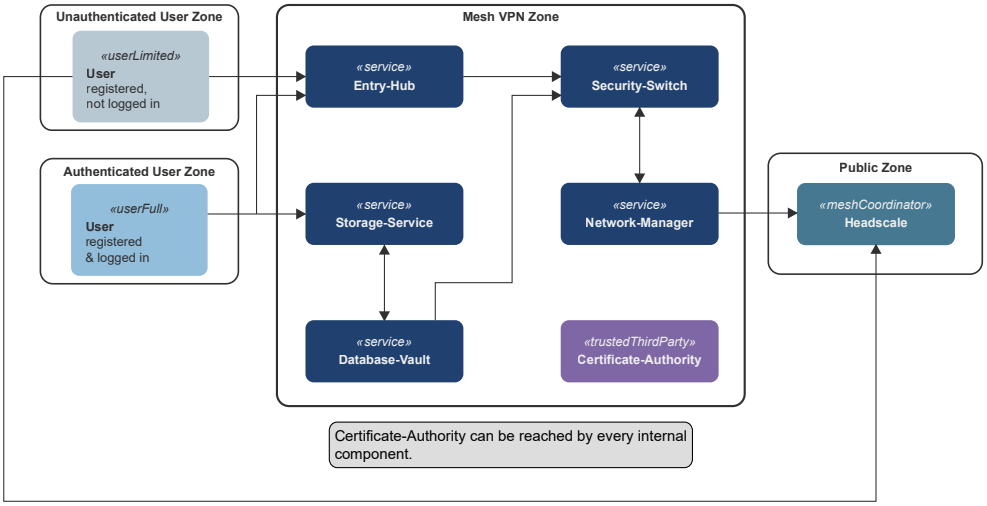


Figure 4.4: Trust zones and reachability on the mesh network.

4.4 PKI and certificate management

RAM-USB uses two certificate authorities, as illustrated in Figure 4.5.

Let's Encrypt, a public CA, signs only the certificate that Entry-Hub presents on its public endpoints. This is because the certificate must be verifiable by an external client that does not yet have access to the system's internal network. A certificate signed by RAM-USB's private Certificate-Authority would not be trusted.

Every other communication instead uses the private Certificate-Authority, based on `smallstep/certificates`.

Every private certificate follows the same bootstrap scheme: each service receives, out-of-band and only once, a single-use bootstrap token, and presents it to Certificate-Authority in exchange for its own initial certificate. Subsequent renewals present the mTLS certificate already held. The token remains a throwaway secret and therefore does not need to be protected once consumed.

A valid X.509 certificate only proves that Certificate-Authority approved that client, not that it is the client expected for that connection. For this reason each service also verifies the `organization` field of the certificate.

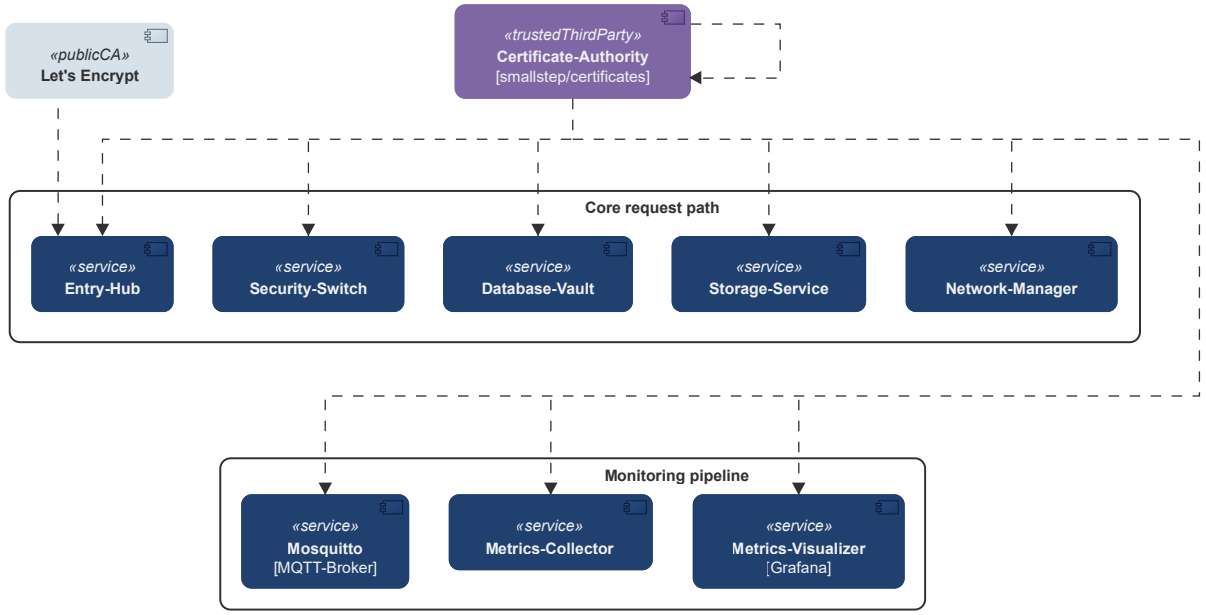


Figure 4.5: Hierarchy of the certificate authorities and signing relationships.

5

Authentication and identity management

5.1 Request lifecycle

A registration or login request passes through three services before becoming a record: Entry-Hub, Security-Switch, and Database-Vault. All three apply the same checks to the input, each one independently of the others.

Validation takes place in two phases. The first concerns the request body: the payload is read up to a limit of 2048 bytes and rejected if it exceeds it, and the JSON decoder rejects any unexpected field.

The second phase concerns the individual fields [18]. The email must be a single, syntactically valid address according to RFC 5322 [28]; the password must be between 8 and 128 characters long and use at least three categories among lowercase letters, uppercase letters, digits, and symbols. At registration, moreover, the SSH public key must be well formed.

If a check fails, the response is HTTP 400 without indicating which check did not pass, the log records the problem without identifying the user, and the request does not proceed inward. The message stays generic because a detailed error would help an attacker more than it helps a user acting in good faith, whose own client has already applied the same checks.

The three services share the same validation library, so repeating the check does not introduce algorithmic diversity, but it guarantees that no service takes for granted that the previous one did its job. In this way, if, for example, a tampered request coming from a compromised service were to reach Database-Vault, the validation would still be performed and would cause the request to be discarded.

Finally, internal errors are translated into HTTP status codes. Table 5.1 summarizes the codes that reach the user, attributing each code to the service that emits it.

Entry-Hub builds only the errors that originate in itself: a failed validation and a call to Security-Switch that went wrong. Everything else it relays back to the client exactly as it arrives from the inside, without modifying it. For this reason, a 401 or a 409 reaches the user despite never being produced by Entry-Hub.

Table 5.1: HTTP status codes returned to the client and their meaning in the system.

Code	Meaning	Issued by
400	Validation failed, without indicating which check did not pass	both
401	Invalid credentials, does not specify whether nonexistent email or wrong password	Security-Switch
403	Explicit refusal from Network-Manager	Security-Switch
409	Email or SSH key already registered, does not specify which of the two	Security-Switch
500	Internal error not otherwise classified	both
502	Downstream service unreachable or returning an unexpected response	both
503	Call to Security-Switch timed out	Entry-Hub
504	Call to Database-Vault or Network-Manager timed out	Security-Switch

The codes that the two services build on their own differ in two respects: the first is 403, which only Security-Switch can emit, because it is the only one of the two that can receive an explicit refusal from an internal service. The second concerns timed-out calls: Entry-Hub responds 503, Security-Switch 504.

In all cases, the response body is sanitized and the error details remain in the internal logs.

5.2 Credential encryption and hashing

Database-Vault treats the data it receives in three different ways:

- the SSH key is public, so it does not require secrecy;
- the email is at the same time a lookup key and personal data to be protected, but it must also be recoverable when needed;
- the password must not be recoverable, not even by whoever administers the database.

The email is normalized to lowercase and reduced to its SHA-256 digest, which becomes the primary key of the record. Normalization guarantees that the same address written with different letter case at login produces the same key as at registration. This hash is deterministic by necessity, because it must enable the lookup. The fundamental limitation of this approach, together with its mitigation, is discussed in Chapter 11.

Alongside the digest that acts as the lookup key, the record keeps an encrypted copy of the email, so that the address remains recoverable. From a master key and a random 16-byte salt, generated for that record, a dedicated key is derived with HKDF-SHA256 [29], with which the email is encrypted with AES-256-GCM [30] using a random 12-byte nonce. Salt, nonce, and ciphertext are stored in the record, while the derived key is zeroed as soon as the cipher has consumed it. Deriving a key for each record, instead of encrypting everything with the master key, reduces the amount of data protected by a single key.

```
func newGCM(masterKey, salt []byte) (cipher.AEAD, error) {
    derivedKey := make([]byte, derivedKeySize)
    kdf := hkdf.New(sha256.New, masterKey, salt, []byte(hkdfInfo))
    if _, err := io.ReadFull(kdf, derivedKey); err != nil {
        return nil, fmt.Errorf("encryption: derive key: %w", err)
    }

    block, err := aes.NewCipher(derivedKey)
    for i := range derivedKey {
        derivedKey[i] = 0
    }
    if err != nil {
        return nil, fmt.Errorf("encryption: new AES cipher: %w", err)
    }

    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, fmt.Errorf("encryption: new GCM: %w", err)
    }

    return gcm, nil
}
```

Code 5.1: Per-record key derivation in `newGCM`, from `encryption.go` [31].

The password, instead, is concatenated with a pepper, read from an environment variable, and passed to Argon2id [32] together with a random 16-byte salt generated on the spot for that record.

OWASP [33] proposes several configurations equivalent to one another, which trade memory for number of passes. RAM-USB starts from the one with the most memory and doubles its passes, thus paying at each verification twice the work that OWASP considers sufficient. It is a cost that makes recovering the passwords from a stolen database impractical. The pepper further complicates an offline attack: unlike the salt, which is stored in the record and only prevents a single guess from applying to several users at once, the pepper does not appear in the database. Without the pepper, no guess is computationally feasible, and a copy of the database is therefore not sufficient to carry out the attack.

The result is stored as a self-describing string in PHC format, the encoding convention that originated from the Password Hashing Competition. Each record therefore carries with it the parameters with which it was created, and verification at login uses them in place of the service's current constants. In this way, changing the parameters for new users does not invalidate already existing records.

Table 5.2 lists the fields of that string, in the order in which they appear in the record.

Table 5.2: Fields of the PHC string in which Database-Vault stores the password hash.

Field	Meaning
`argon2id`	Algorithm variant
`v=19`	Argon2 version: the byte `0x13` fixed by RFC 9106, printed in decimal
`m=47104`	Working memory in KiB, i.e. 46 MiB
`t=2`	Number of passes over memory
`p=1`	Degree of parallelism
salt	Random 16-byte salt, generated for that record
digest	32-byte hash produced by Argon2id

The master key and the pepper both reside in environment variables, with no backup or rotation procedure. The risk is discussed in Chapter 12.

Scheme 5.1 summarizes the transformations just described, with the following symbols:

- e: the received email;
- p: the received password;
- k_{SSH}: the SSH public key;
- π: the pepper;
- MK: the master key;
- s, n, s_p: respectively the HKDF salt, the AES-GCM nonce, and the Argon2id salt, generated for that record;
- $\xleftarrow{R}$: a uniform random draw from the indicated set.

1. $s,\, s_p \xleftarrow{R} \{0,1\}^{128}, \quad n \xleftarrow{R} \{0,1\}^{96}$
2. $k_h = \text{SHA-256}(\text{lower}(e))$
3. $k = \text{HKDF-SHA256}(MK,\, s)$
4. $c = \text{AES-GCM}_k(n,\, e)$
5. $h = \text{Argon2id}(p \,\|\, \pi,\, s_p)$
6. $r = (k_h,\, s,\, n,\, c,\, \text{PHC}(h,\, s_p),\, k_{SSH})$

Scheme 5.1: Transformations applied to the credentials before the record is written.

5.2.1 Password strength

The validation rules of Section 5.1 delimit the space of admissible passwords, and from that space one can derive, in the best case, how much uncertainty the policy can guarantee against whoever tries to guess a password, that is, how many possibilities they must consider, not knowing which one was chosen.

How much that uncertainty is worth, however, depends on how the password was chosen. In the best case, in which it is produced by a pseudorandom generator drawing from the entire admissible alphabet, every admissible password is equiprobable and the uncertainty is therefore maximal. In the worst case, in which it is constructed by a person, some passwords are far more probable than others;

for example, `Password1!` is chosen much more often than a random sequence of the same length. The value computed here is that of the best case, that is, an upper bound.

In this case, then, the computation reduces to counting the distinct values that the policy admits, starting from the cardinality of the four categories:

$$\begin{aligned} n_1 &= 26 && \text{lowercase} \\ n_2 &= 26 && \text{uppercase} \\ n_3 &= 10 && \text{digits} \\ n_4 &= 33 && \text{symbols} \\ N &= \textstyle\sum_i n_i = 95 && \text{characters available in total} \end{aligned}$$

The constraint of at least three categories excludes the strings that use at most two of them, so to count the valid passwords it is easier to first count those that are not valid: one starts from all N^L possible strings and subtracts those that use a single category and those that use exactly two.

The first group is immediate, because the strings written with category i alone are n_i^L. For the second, it is not enough to sum $(n_i + n_j)^L$ over all pairs, because that value counts the strings written with those two alphabets only and therefore also includes those of i alone and of j alone: subtracting n_i^L and n_j^L leaves the strings that actually use both categories. By the inclusion-exclusion principle, the number of valid passwords of length L is therefore

$$V(L) = N^L - \sum_{i=1}^{4} n_i^L - \sum_{i<j} \left[(n_i + n_j)^L - n_i^L - n_j^L \right].$$

An attacker lacking any information must consider all $V(L)$ admissible candidates. That number is customarily expressed in bits, that is, as an exponent of 2: b bits correspond to 2^b candidates. If the password is drawn uniformly, the attacker's uncertainty is therefore $\log_2 V(L)$ bits.

For the minimum admissible length, $L = 8$, one obtains $V(8) = 6.27 \cdot 10^{15}$, that is, $\log_2 V(8) \approx 52.48$ bits.

The comparison with the unconstrained space shows how much each of the two rules weighs. Without the category constraint, all eight-character strings over the alphabet would be admissible, namely $N^8 = 95^8 = 6.63 \cdot 10^{15}$, that is, 52.56 bits. The category constraint therefore reduces the entropy by 0.08 bits. Each additional character, instead, increases the entropy by $\log_2 95 \approx 6.57$ bits.

From these figures it might seem that the constraint simplifies the work of a malicious actor instead of countering it, and this is true, albeit to a negligible extent, in the case in which passwords are created by pseudorandom generators. The purpose of that rule, however, is not to increase entropy, but rather to constrain the choice of a person, who tends to stop at lowercase letters only or at letters and digits. How much it is worth in that role cannot be derived from this computation.

Whether the two rules should coexist, or whether it would be better to replace the category constraint with a greater minimum length, therefore remains an open question, taken up among the future work in Chapter 12.

How well that space holds up, however, also depends on the cost of each guess, that is, on the Argon2id parameters (Table 5.2).

5.3 Registration

When the request reaches Database-Vault and has been revalidated, the service computes the three transformations of the previous section and tries to write the record in a single atomic transaction [34]. Two uniqueness constraints protect the table, one on the email hash and one on the SSH public key: if either of them is violated, the registration is rejected with HTTP 409, without saying which of the two caused the conflict. That code, however, tells an unauthenticated sender that one of the two submitted values was already present, and whoever submits a freshly generated SSH key knows that the only other candidate is the email. Chapter 11 analyzes this problem and presents a possible mitigation.

At this point the record exists, but the user does not yet have a space in which to write. Database-Vault then generates a POSIX username of the form `user<xxxxxx>`, where the six digits are random characters from a base-36 alphabet, and asks Storage-Service to create the corresponding account. How Storage-Service creates and isolates it is the subject of Chapter 6.

If that creation fails, the record just written is deleted. If the rollback also fails, the service reports it with a dedicated error.

When instead both steps succeed, Database-Vault informs Security-Switch that the registration was successful, and the latter proceeds with the orchestration described in the concluding section of this chapter.

```
    rollbackCtx, cancel := context.WithTimeout(context.WithoutCancel(ctx), rollbackTimeout)
    defer cancel()

    if delErr := store.DeleteUser(rollbackCtx, input.EmailHash); delErr != nil {
        return Result{
            Outcome: OutcomeFailed,
            Err: fmt.Errorf("%w: posix creation error: %w; delete error: %w", ErrOrphanedRecord,
                err, delErr),
        }
    }
```

Code 5.2: Compensating rollback after a Storage-Service failure, from `registration.go` [35].

Figure 5.1 summarizes the entire sequence, from the client's request to the response. The last two exchanges, those toward Network-Manager, are the subject of the concluding section of this chapter.

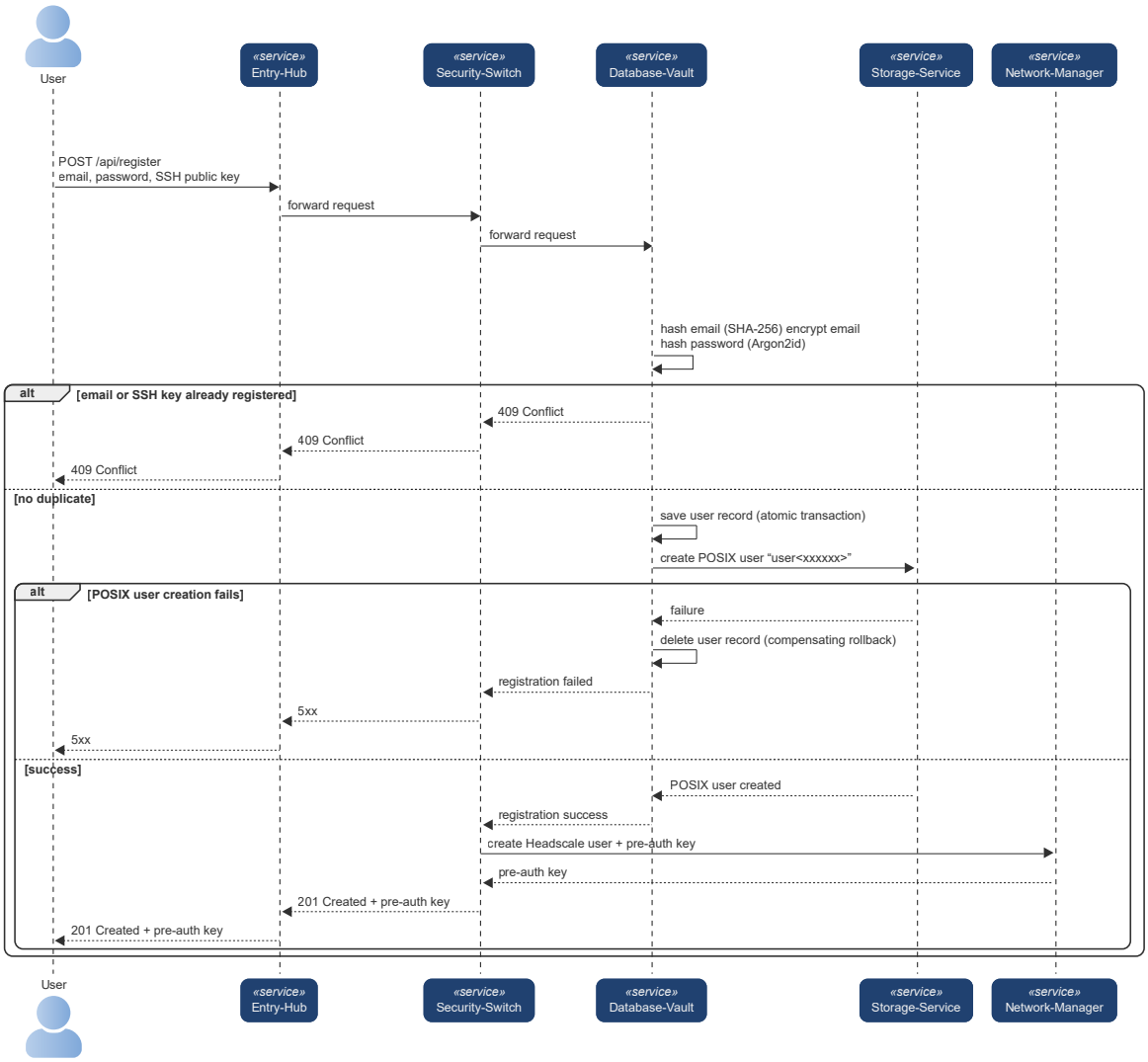


Figure 5.1: Registration sequence (UC-01).

5.4 Login

During login, Database-Vault recomputes the SHA-256 digest of the received email and uses it to look up the record.

Salt and cost parameters are extracted from the record's PHC string. Argon2id is recomputed with those values and with the pepper, and the resulting digest is compared with the stored one through a constant-time comparison, which therefore does not allow one to infer how many leading bytes of the two values match.

A nonexistent email and a wrong password must produce the same response, and must not be distinguishable even in the logs. The two paths therefore converge on a single sentinel error, and the content of the original error is discarded [36].

Making the response and the log identical, however, is not enough to make the two cases indistinguishable, because a third observable difference remains: time. Argon2id, with the parameters of the previous section, costs tens of milliseconds, and if it were executed only when a record actually exists, an

attacker could distinguish the two situations by measuring the response latency. The solution adopted is to spend the same time even when there is nothing to verify.

```
func VerifyDummy(password, pepper []byte) {
    _, _ = VerifyPassword(password, pepper, dummyHash())
}
```

Code 5.3: Dummy verification used on the rejection paths, from `hash.go` [37].

The function recomputes Argon2id against a dummy hash, built only once per process with the same cost parameters as the real records, and discards the result. It is called on every rejection path, including the one in which the lookup itself fails, so that not even an infrastructure failure becomes easily measurable from the outside.

Every uncertainty leads to the same outcome. If the verification cannot complete, for example because the stored hash is malformed, the response remains 401, according to the fail-secure principle [22]. In the logs that case remains distinguishable, because an operator needs to know whether it is a corrupted record, a careless user, or a genuine attack.

Figure 5.2 summarizes the complete login sequence, including the branch in which the two causes of rejection converge on the same response.

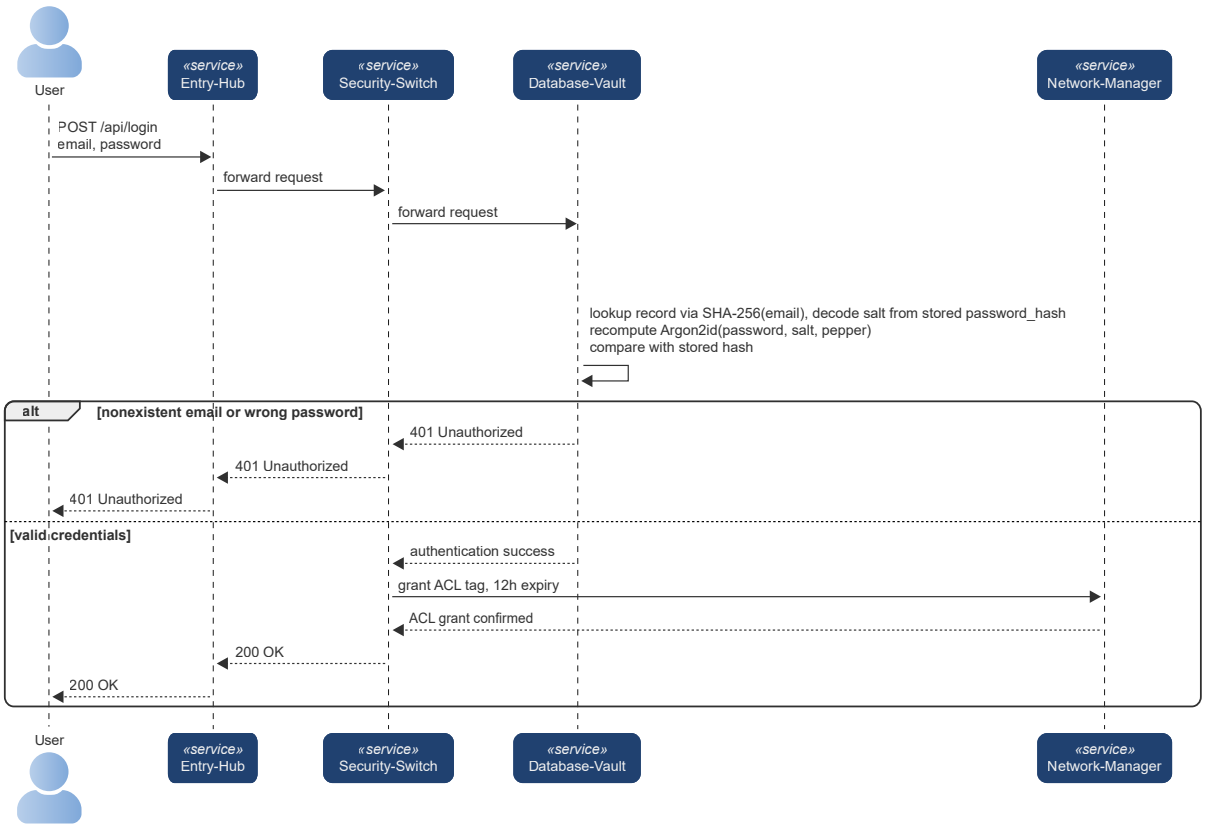


Figure 5.2: Authentication sequence (UC-02).

5.5 Post-authentication orchestration

Authenticating a user, however, still does not give them access to anything. Access, in this system, is a property of the network. Security-Switch asks Network-Manager to modify the network rules at two different moments.

After a successful registration, it requests the creation of the user's identity on the mesh network and of a pre-auth key, which travels back up the chain to the client inside the 201 response and which allows the user's device to join the private network.

After a successful login, instead, it requests twelve-hour temporary access to Storage-Service.

Both steps, however, grant reachability, not read access: registration puts the device in the network, login allows it to reach Storage-Service, but what opens the backups is the SSH private key, which never leaves the user's device. Knowing the user's credentials is therefore not a sufficient condition to access their backups.

How Network-Manager creates the mesh identity, applies the ACL tag and manages its expiry is the subject of Chapter 7.

6

File storage and backup

6.1 SFTP-only access model

Storage-Service stores the users' backups and is the only component the client talks to directly after authentication. Indeed, once authenticated, the user becomes able to reach it on the private network and is therefore ready to manage their backups.

Before transmitting them, however, the user's client, through Restic [20], deduplicates, compresses, and encrypts the user's backups. Storage-Service therefore receives a sequence of encrypted blocks that it is not able to decrypt. When initializing the Restic repository, the client draws 32 bytes from `crypto/rand`, encodes them in base64 and uses them as the repository password [38]. From that password Restic derives the key that opens the file in which it keeps the actual encryption key, generated at random when the repository is created [39]: it is the latter that protects the blocks.

This approach was designed to prevent the user from choosing a weak password. The client saves the repository password on the user's device, with the same permissions as the SSH private key. Its secrecy, like the responsibility of copying the keys to a safe place, is left to the user. Should even a single one of the credentials be lost, the user would have no way to access the content of their backups.

The fact that the users' data is encrypted, however, does not mean that it should be reachable by anyone. The only way to reach the blocks is in fact an authenticated SFTP session, inside the private mesh network. On that session, the only possible operation is transferring files, and the only credential accepted is the SSH key [40].

Left out are the other ways of authenticating and the other uses that SSH normally allows, such as executing a command, opening a shell, or using the connection as a relay to reach other machines.

Moreover, only the accounts created by the service can log in, because the server accepts only usernames of the form `user*` and root access is explicitly disabled.

OpenSSH's default list of cryptographic algorithms [40] is deliberately broad to guarantee compatibility with clients that do not support the modern and more robust algorithms, but here that compatibility is not needed, and only risks weakening the system; therefore, the cryptographic algorithms are also listed by hand [41].

6.2 Isolation and dynamic chroot

Isolation between users rests on an operating system mechanism called chroot, which confines a process inside a subdirectory: from that moment on that directory is, for the process, the root of the filesystem, and outside of it there is no longer any path that the process can name.

At registration, Storage-Service prepares for the new user the POSIX account that will inhabit that space. The account has no traditional home directory and its shell is `/usr/sbin/nologin`, a program that merely refuses access.

For every POSIX user whose name matches `user*`, the rules specified in Code 6.1 are also applied.

```
Match User user*
    ChrootDirectory /storage/%u # confines the session to the user's subtree
    ForceCommand internal-sftp # enforces SFTP in place of the command requested by the client
```

Code 6.1: The two per-user directives, from `sshd_config` [41], with added comments.

Alongside the account, the directory structure illustrated in Figure 4.2 is created. The root of the confinement belongs to root, while the `data` subdirectory belongs to the user and is the only place where they can write. Ownership by root is a requirement of the SSH server, which refuses to confine a session in a directory writable by anyone else.

The result is that a connected user sees as the root of their filesystem the directory that belongs to them, and the other users' directories are neither reachable nor listable in any way.

Normally an SSH server looks for the keys that authorize a connection in a file inside the user's home, where the keys accepted for that account are listed. Here that file is not consulted. In its place, at every connection attempt, the server runs a dedicated binary, passing it the username being presented. The binary runs under a system identity that has no other duties on the machine and asks Database-Vault for that user's current public key over an mTLS channel.

Whatever goes wrong produces the same outcome: the binary exits without printing anything [42]. The server receives an empty list of keys and denies the connection, following the fail-secure principle [22]. There is no other place outside Database-Vault in which the SSH public keys are stored, so a malfunction in Database-Vault prevents users from starting SFTP sessions.

6.3 Backup and restore

The backup takes place entirely on the user's device. The client invokes Restic, indicating as the repository the writable subdirectory inside the user's own confined space, reached via SFTP, and requires it to authenticate with the private key generated at registration [43], which has never left the machine and which no component of the system has ever seen.

The path given for the repository is already relative to the root of the confinement.

The client resolves Storage-Service through MagicDNS on the mesh network, and the resolution succeeds only as long as the temporary grant given by the last login is active: once that has expired, the user's node no longer sees the service and the connection does not even start. Chapter 7 describes how that grant is given and revoked.

Figure 6.1 summarizes the sequence, from the SFTP connection to the written data.

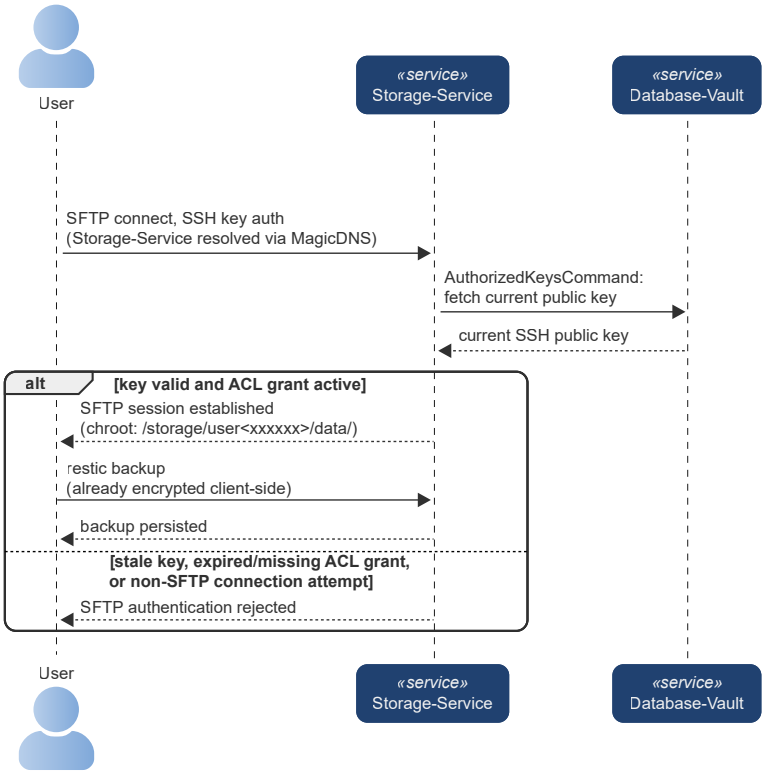


Figure 6.1: Sequence for backing up a file (UC-03).

Restore follows the same path as backup; the only difference is the command issued to Restic, which downloads the requested blocks and decrypts them on the user's device.

7

Networking and mesh coordination

7.1 Network policies and reachability boundaries

The trust zones of Figure 4.4 describe who can contact whom. The reachability boundaries that derive from them are enforced by a network policy that Network-Manager publishes on Headscale.

Every node of the mesh network, be it a service or a user's device, carries one or more labels called tags. The reachability rules are written on tags rather than on IP addresses, so as to survive a change of a service's address, for example because it restarts on a different node.

Network-Manager collects all the rules into a single document and delivers it to Headscale only once, at startup [44].

Headscale in turn translates it into a packet filter: it tells every node that joins the network only about the peers that the rules allow it to contact, and then sends it the subsequent updates concerning those peers. In this way, a service that by policy must not contact another one is not even able to see it in the network.

The rules, moreover, name only the party that opens the connection, so replies travel without a rule in the opposite direction. In this way, for example, Security-Switch can contact Database-Vault, and the latter can reply to the request, but is not able to initiate communication with it.

7.2 Access grant lifecycle

The two rules that concern the user are shown in Code 7.1. The tag `TagStorageAccess` is temporary and is assigned at login; `TagMeshMember` is permanent and is assigned when the device joins the network.

```
{ // only an authenticated user, and Database-Vault, reach Storage-Service
    Action: "accept",
    Src: []string{TagStorageAccess, TagDatabaseVault},
    Dst: []string{dstAny(TagStorageService)},
},
{ // every registered user reaches Entry-Hub
    Action: "accept",
    Src: []string{TagMeshMember},
    Dst: []string{dstAny(TagEntryHub)},
},
```

Code 7.1: The two rules that delimit the user's zones, from `policy.go` [44], with added comments.

At registration, Network-Manager creates for the new user an identity on Headscale and a pre-auth key, that is, a single-use credential with which the device presents itself to the coordinator to join the network [45]. The key is valid for fifteen minutes, and travels back up the chain of services to the user inside the registration response.

The key is created already associated with the membership-only tag. The device that uses it therefore joins the network and gains reachability to Entry-Hub's private network interface.

At login Network-Manager looks up the user's node and adds the second tag to it, then records an expiry twelve hours from that moment [46].

Looking up the user's node, however, requires Network-Manager to keep a small SQLite database containing the numeric identifier of the pre-auth key bound to the hash of the user's email, with the schema shown in Code 7.2.

```
CREATE TABLE IF NOT EXISTS mesh_users (
    email_hash TEXT PRIMARY KEY,
    pre_auth_key_id INTEGER NOT NULL
);
```

Code 7.2: Schema of the email-to-pre-auth-key mapping table, from `meshusers.go` [47].

The tag expiries are also persisted in the database, to prevent a restart of the service from turning a twelve-hour grant into a perpetual grant. The schema of the table that holds them is shown in Code 7.3.

```
CREATE TABLE IF NOT EXISTS grants (
    email_hash TEXT PRIMARY KEY,
    node_id INTEGER NOT NULL,
    tag TEXT NOT NULL,
    expires_at INTEGER NOT NULL
);
```

Code 7.3: Schema of the access grant table, from `store.go` [48].

A periodic process, every five minutes, looks for expiries that have passed and revokes the expired grants [49]. The row is deleted from the database before calling Headscale to remove the tag, and the deletion is conditional on the exact expiry just read. If in the meantime a login has renewed the grant, the expiry in the table no longer matches, so the deletion touches no row and the sweep stops without

contacting Headscale. Calling Headscale first and deleting afterward would instead have risked also deleting the row just rewritten by the concurrent login, taking away from the user a grant that had just been given.

Figure 7.1 summarizes the states that a user's node goes through.

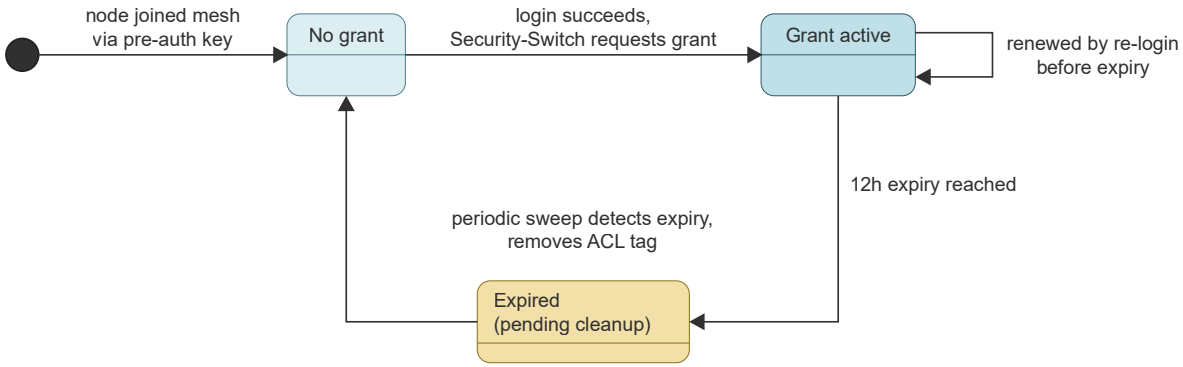


Figure 7.1: Lifecycle of the access grant associated with a user's node.

7.3 The coordinator paradox: Headscale as a standalone service

The Headscale documentation states that running Headscale on a machine that belongs to the coordinated network is not recommended, because it can cause problems with name resolution through MagicDNS [50].

The coordinator must therefore remain outside the network it coordinates, and three consequences follow from this.

The first is that it cannot share a virtual machine or a container with any other component, because all the others are members of the mesh.

The second is that the traffic with which Network-Manager creates the keys and assigns the tags cannot be protected by its placement in the network, as is the case for every other internal call. It is the only call between components of the system that crosses the public network, and the only one whose protection depends entirely on mTLS and on the Headscale API key.

The third is that the coordinator must be publicly reachable, because a device that has not yet joined the network must be able to contact it the first time in order to join the network.

Headscale, however, has a limitation that is relevant for RAM-USB: it offers no way to separate the coordination protocol, which must be reachable by everyone, from the administrative API, which must be reachable only by Network-Manager. The separation was entrusted to a reverse proxy, which distinguishes the two surfaces based on the path of the request [51]. Coordination requests pass without any check, while administrative ones are accepted only if accompanied by a certificate issued by the private Certificate-Authority, in addition to an API key that Headscale itself requires. The administrative traffic is defended by two independent credentials because it is one of the most sensitive surfaces.

The certificate presented by the proxy is instead issued by a public CA, for the same reason that Entry-Hub's is.

7.4 CG-NAT and the need for a dedicated public endpoint

The central pool of not-yet-assigned IPv4 addresses was exhausted in 2011 [52], and to keep connecting new customers many Internet Service Providers have several subscribers share a single public IPv4 address. The home router no longer receives a public address, but one from the `100.64.0.0/10` block, valid only inside the operator's network and not routable on the internet [53]; it is a NAT managed by the operator that translates it into the shared public address. This type of NAT is called Carrier-Grade NAT [54]. Since that NAT is not under the customer's control, the customer cannot open inbound ports on it by themselves, and in general a server hosted at home is not reachable from the internet [55]. RAM-USB is designed so that it can also be hosted by home users, who therefore might not have neither a public IPv4 address nor IPv6 connectivity. For this reason Headscale and Entry-Hub each reside on their own Virtual Private Server, separate from the rest of the system. Chapter 10 describes the actual placement of each component.

8

Observability and metrics

8.1 Publishing the metrics

Operational metrics can typically reach whoever observes them in two ways: either someone else periodically queries every service (*pull*), or every service sends them on its own initiative (*push*). RAM-USB adopts the second: every service publishes its own metrics once a minute, on its own dedicated MQTT topic to the Mosquitto broker [56]. The choice stems from the fact that a *pull* model would require Metrics-Collector to open a connection to each of the six services, widening the mesh reachability rules discussed in Chapter 7. Because the services publish on their own initiative, they are instead the ones that open the connection to the broker. From there, Metrics-Collector reads the metrics and persists them in a database.

Operational metrics do not contain user data, but the system's metadata must be protected with the same care; for this reason the communications take place inside the private network and are protected by mTLS, with certificates issued by the internal CA.

Every publication carries as its body a six-field payload, shown in Code 8.1. The metrics in the payload refer to the minute in which they were sent.

```
type Payload struct {
    Service string `json:"service"`
    Timestamp string `json:"timestamp"`
    RequestCount int64 `json:"request_count"`
    ErrorCount int64 `json:"error_count"`
    AverageResponseTimeMs float64 `json:"average_response_time_ms"`
    ActiveConnections int64 `json:"active_connections"`
}
```

Code 8.1: Structure of the metrics payload, from `payload.go` [57].

The publications all use *QoS 1* (at least once), in the MQTT classification [58]: the broker guarantees that the message arrives at least once, at the cost of a possible duplicate. *QoS 0* (at most once) is ruled out because RAM-USB cannot afford to lose metrics silently; *QoS 2* (exactly once) is ruled out because the extra guarantee it offers (the absence of duplicates) is not needed here, since a duplicate only results in two overlapping points at the same instant, invisible on the chart.

8.2 Reception and consistency check

The published message must reach a single recipient. This property is enforced by the Mosquitto ACL, which ensures that every service can only write to its own topic, while Metrics-Collector can read from the `metrics/#` topics without being able to write to them.

That ACL, however, does not prevent a compromised service, while staying within its own legitimate write permission, from publishing a payload whose declared `service` field is a name different from its own. Metrics-Collector therefore revalidates every message independently, deriving the expected service from the topic the message arrived on and comparing it with the one declared in the payload. On any discrepancy it discards the message and records it in the logs.

8.3 Time-series persistence

Metrics-Collector persists the metrics in TimescaleDB [25], an extension of PostgreSQL designed for a workload made almost entirely of time-ordered writes.

The `metrics` table has no primary key, since a row of metrics has no natural identity. Moreover, TimescaleDB [25] does not require one. It is organized in chunks by time intervals on the `time` column: the two background policies in Code 8.2 act chunk by chunk, based on their age. The first drops every chunk older than thirty days; the second, after seven days, moves the chunks into a compressed columnar format, grouped by service and ordered by descending time.

```
SELECT create_hypertable('metrics', by_range('time'));

SELECT add_retention_policy('metrics', drop_after => INTERVAL '30 days');

ALTER TABLE metrics SET (
    timescaledb.enable_columnstore,
    timescaledb.segmentby = 'service',
    timescaledb.orderby = 'time DESC'
);
CALL add_columnstore_policy('metrics', after => INTERVAL '7 days');
```

Code 8.2: Automatic retention and compression policies of the `metrics` table, from `000001_create_metrics_table.up.sql` [59].

8.4 Metrics visualization

An administrator checks the health of the system, without that consultation ever touching the users' data since, by construction, the metrics do not contain any. Grafana [26] reads from TimescaleDB directly.

To contact TimescaleDB, in addition to reachability on the mesh network and Grafana's mTLS certificate, a password is also required, following the same defense-in-depth principle already seen for Headscale's administrative API in Chapter 7.

The dashboard shows three panels, each a direct query on the `metrics` table: average response time, throughput, and active connections.

Figure 8.1 summarizes the entire pipeline just described, from publication on the MQTT topic to the dashboard.

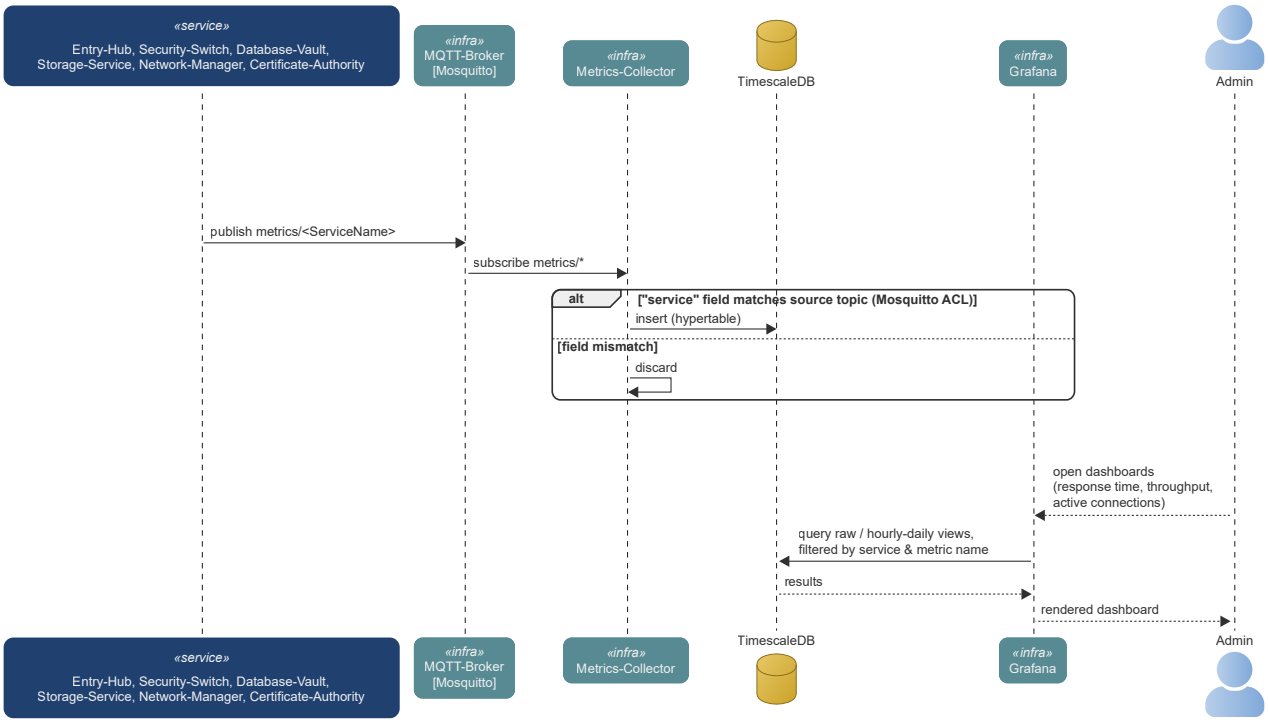


Figure 8.1: Observability pipeline, from the publication of the metrics to the Grafana dashboard.

9

Testing and validation

9.1 The V-model and the four test levels

For the testing of RAM-USB the V-model was adopted, an extension of the waterfall model in which each specification phase is paired with a corresponding verification phase, written hand in hand with the specification.

This approach was adopted because RAM-USB was deliberately treated as a safety-critical system, in which a defect in production has serious consequences for the users of the system. It was therefore essential that an error in the specification be detected well before implementation and that testing be applied at several specification levels.

Table 9.1 shows the correspondence between the requirement category and the test level that verifies it, along with the technique used at that level.

Table 9.1: Verification level associated with each specification level.

Specification level	**Verification level**
Component requirement	Unit testing with Go's `testing` package
Use case	Integration testing, bottom-up with drivers and stubs
Non-functional requirement	System testing, on the whole stack
User requirement	Acceptance checklist, walked through by hand on the corresponding use case

9.2 Bottom-up integration strategy

Integration starts from the innermost component and proceeds outward, in the order Database-Vault, Storage-Service, Security-Switch, Network-Manager, Entry-Hub.

The choice to start from Database-Vault is dictated by the fact that it is at the same time the component with the fewest outgoing dependencies and the one on which the largest number of functional requirements is concentrated. Building it first, with drivers standing in for the components that invoke it and a stub standing in for Storage-Service, the only component it invokes, ensures that the most sensitive component is also the one verified the longest.

9.3 TDD: entry and exit criteria

The entry criterion is that a test exists for every requirement, and that any failure of that test occurs before the implementation of the requirement under test.

An example: requirement DV-F-15 requires that a nonexistent email and a wrong password produce the same login outcome, indistinguishable from the outside.

The test that verifies it, `TestLogin_NonexistentEmailAndWrongPassword_AreIndistinguishable` [60], calls `Login` with the two different credentials and requires that the outcomes match exactly.

A test that has never been seen to fail, perhaps written after the code and passing on the first try, offers no guarantee that it verifies anything: it might compare two values that are equal by construction, or not call the function it is supposed to test at all. For this reason, the test is written before the body of the `Login` function actually exists, and it fails, providing proof that the test can recognize a wrong outcome.

Only after the test has failed does it make sense to write the complete body of `Login`, and only then does the transition from failed test to passed test become useful information.

Writing tests first also helps to disambiguate and clarify requirements. Indeed, if a test cannot be written for a requirement, the requirement is unclear or ambiguous and consequently must be reworded more precisely.

There are three exit criteria, and they depend on the code under test. The first concerns only the validation library, which requires more thorough tests because it is considered particularly sensitive.

- in the validation library shared by Entry-Hub, Security-Switch, and Database-Vault (Section 5.1), every branch of every `if` or `switch` that decides whether to accept or reject a field must be traversed by at least one test case;
- no test that has already passed on the already integrated chain of components may fail;
- a change reruns not only the test written specifically for it but the entire test suite.

9.4 Coverage and gaps in the test plan

`go test -cover` measures, for each package of the codebase, the percentage of statements executed at least once: `pkg/validation`, for example, reaches 98%, because that is where the application logic lives, while the `cmd` packages, which start each service, stay at 10–20%, because verifying them is the job of integration and system testing.

The exit criterion mentioned above for `pkg/validation` is a stricter measure that adds to this one: it is not enough to execute almost all statements, every single branch of every `if` or `switch` must have been taken by at least one test.

The test plan, however, is not free of limitations: No test is planned for either protecting backups from tampering or rotating the master key, currently out of scope, but essential in a project of this kind. Load and stress testing, moreover, are planned but not yet designed. There are therefore no reliable benchmarks of the system's behavior. Every end-to-end test on the host involved a single user with a single small file in the backup, so neither concurrency nor mutual isolation between two users has been put to the test.

9.5 Automated pre-commit checks

In addition to the entry and exit criteria, an automated verification chain runs before every commit [6]. `golangci-lint` [61] is a runner that aggregates multiple linters in a single pass; the RAM-USB configuration enables fifteen of them in total.

Among these, `gosec` [62] looks for known security patterns in the code, and has been present since the early stages of implementation.

A second extension introduced four linters targeting unreleased resources and mishandled errors:

- `bodyclose` [63] flags an HTTP response whose body is never closed, which therefore leads to a resource leak;
- `sqlclosecheck` [64] does the same for SQL rows and statements left open;
- `contextcheck` [65] detects a function that does not propagate the `context.Context` received from the caller;
- `errorlint` [66] checks that errors are compared in the correct way.

These are complemented by Go's race-condition detector, invoked with `go test -race`.

The chain is completed by `govulncheck` [67], which compares the dependencies against a database of known vulnerabilities but reports a problem only if the code actually reaches the vulnerable function, and by two scanners designed to overlap with each other: `gitleaks` [68], which looks for secrets already committed to the repository history, and `trivy` [69], invoked in filesystem scan mode (`trivy fs .`), which checks both dependencies and secrets independently of the two previous tools.

The redundancy of the last two is the application of the defense-in-depth concept to testing.

9.6 Results: completed verifications

The most significant campaign was run on real hardware, a Proxmox host with the components deployed according to the placement described in Chapter 10.

The four use cases passed: registration and login succeeded, the backup completed and the restore returned a file identical to the original.

Table 9.2 shows the properties checked on the running system.

Table 9.2: Properties verified on the running system.

Property	**Observed result**
Hashing parameters	The string stored in the record shows `argon2id`, version 19, memory 47104 KiB, two passes, parallelism one
Filesystem isolation	Confinement root owned by root with permissions 711, user subdirectory 700
Identity on the mesh	Every node is owned by the synthetic user `tagged-devices`, never by the account created for the person
Grant duration	Expiry recorded at exactly twelve hours from the moment of login
Reachability boundaries	No node lists among its peers a component that the policy forbids it from contacting
Restart survival	A service that is stopped and restarted comes back into service reusing the identity on disk, without presenting the already consumed single-use token again

During the execution of the complete test plan, several defects emerged that had not been caught by the earlier unit tests [70].

Two concerned the client and prevented backup entirely. The first was the SSH port: the client connected to the default one, while the server listens on another. The second was the absence of any host key policy: `ssh`, invoked by Restic without a terminal, had no `known_hosts` to check and no way to ask for confirmation, and refused the connection instead of asking, so the first backup on every new machine always failed. A third concerned the configuration, because the client has a single address for a service that exposes two on different listeners. A fourth was a usage string documenting a syntax that the argument parsing cannot accept.

A fifth was more serious: the mTLS identity was not preserved from one startup to the next, so no service survived the restart of its own container, because it retried the bootstrap with a single-use token already spent.

Other small logic and implementation errors, too numerous to be reported in this thesis, were detected and fixed thanks to the tests.

10

Deployment

10.1 Containerization and s6-overlay supervision

Some RAM-USB services must run several independent processes in the same container and therefore need a supervisor that starts them in the right order and restarts them when they go down. The one used by RAM-USB is `s6-overlay` [71]. Each supervised process is a directory under `s6-rc.d/`, declared as type `longrun`; its dependencies are files inside a `dependencies.d/` subdirectory, one for each process that must be started: if, for example, process A must wait for process B to be ready, `A/dependencies.d/` contains an empty file named `B`. The startup order therefore arises from a graph declared between directories. Storage-Service is the most complex case: the same Docker container is shared by four processes [72]: `tailscaled`, the actual mesh client; `sshd`, the SFTP server (Chapter 6); `identity-provisioner`, which keeps the X.509 certificate for mTLS up to date; and the Go binary of Storage-Service itself, which exposes the internal HTTP API. Each `longrun` process also has a `run` file and a `finish` file. The first is the script that launches the actual binary [72]. If that process terminates, the supervisor relaunches it from scratch; the second is optional and is used to delay the relaunch by a few seconds, so as to prevent an irreversible startup error from turning into a continuous crash loop.

Not all services, however, need this complexity. Entry-Hub, for example, does not supervise any secondary process: its membership in the mesh lives in a separate Docker sidecar, `entry-hub-mesh`, which shares its network namespace. The following section explains how this difference is tied to the choice of each service's base image.

10.2 Why not all containers are distroless

A distroless image [14] has neither a shell nor a package manager, which substantially reduces the number of known critical vulnerabilities in containers [16]. A supervisor such as `s6-overlay`, however, needs a shell for its own startup scripts. Any service that must run more than a single Go binary therefore cannot be distroless. Consequently, the only distroless service is Entry-Hub [73], while the other six use `debian:bookworm-slim` or `alpine`, as shown in Table 10.1.

Table 10.1: Base image of each service and the processes sharing its container.

Service	**Base image**	**Shares the container with**
Entry-Hub	distroless	none (mesh in a separate sidecar)
Security-Switch	debian:bookworm-slim	`tailscaled`
Network-Manager	debian:bookworm-slim	`tailscaled`
Storage-Service	debian:bookworm-slim	`tailscaled`, `identity-provisioner`, `sshd`
Database-Vault	`postgres:17` (Debian)	`tailscaled`, `postgres`
Metrics-Collector	`timescaledb` (Alpine)	`tailscaled`, `timescaledb`
Certificate-Authority	`step-ca` (Alpine)	`step-ca` (mesh in a separate sidecar)

Extending distroless to more services or choosing distributions other than those listed in Table 10.1 is an issue that has not yet been addressed.

10.3 Local Docker vs Proxmox VE: two levels of isolation

It would be natural to think of Docker and Proxmox VE [74] as two alternatives for the same isolation need. In reality they operate at two different levels. Docker is a containerization engine: it isolates a process at the operating system level, sharing the kernel with the host, so that each container sees its own filesystem, its own network, and its own process tree without the cost of a second kernel [75]. It thus guarantees portability across operating systems and per-service isolation, identical in development and in production.

Proxmox instead adds, on top of each Docker container, the isolation boundary of a KVM virtual machine, with its own kernel, or, alternatively, that of an LXC system container, lighter but sharing the kernel with the host. Each service therefore runs inside its own Docker container, but that Docker container in turn runs inside its KVM virtual machine or inside its LXC container. Choosing Proxmox also brings another advantage: it paves the way for live migration of the KVMs and, with more nodes, for high availability.

Storage-Service, Database-Vault, and Network-Manager are assigned to KVM guests, while the remaining services are assigned to unprivileged LXC containers, in which the container's root user is therefore mapped to a non-root user of the host. Entry-Hub and Headscale stay outside Proxmox entirely, each on its own dedicated public VPS, for the reasons explained in Chapter 7.

The choice between KVM and LXC depends, for almost all services, on the level of isolation they require. LXC containers share the kernel with the host system, so they are not fully isolated, but they need very few resources, so the host can run several of them at the same time. For this reason they are the default choice. The services that hold user data, manage network policies, and store backups need greater isolation, because their compromise could cause irreparable damage. Storage-Service also needs to run `setuid` and `chroot` to isolate POSIX users (Section 6.2): doing so inside an LXC container, itself already a namespace, would add a second level of UID mapping. A KVM avoids stacking two isolation mechanisms that do not compose cleanly [76].

10.4 Distributed topology

The repository contains eleven separate Compose files under `deployments/compose/` [77]. Ten correspond to the internal components (Chapter 4); the eleventh, `tailscale-test.yml`, supports testing. Containerization of the client is instead left to future work. Each component's file remains independent of the others because every service is designed to be started, stopped, or moved to a different Proxmox node, into a different KVM guest or LXC container.

Figure 10.1 summarizes the production topology.

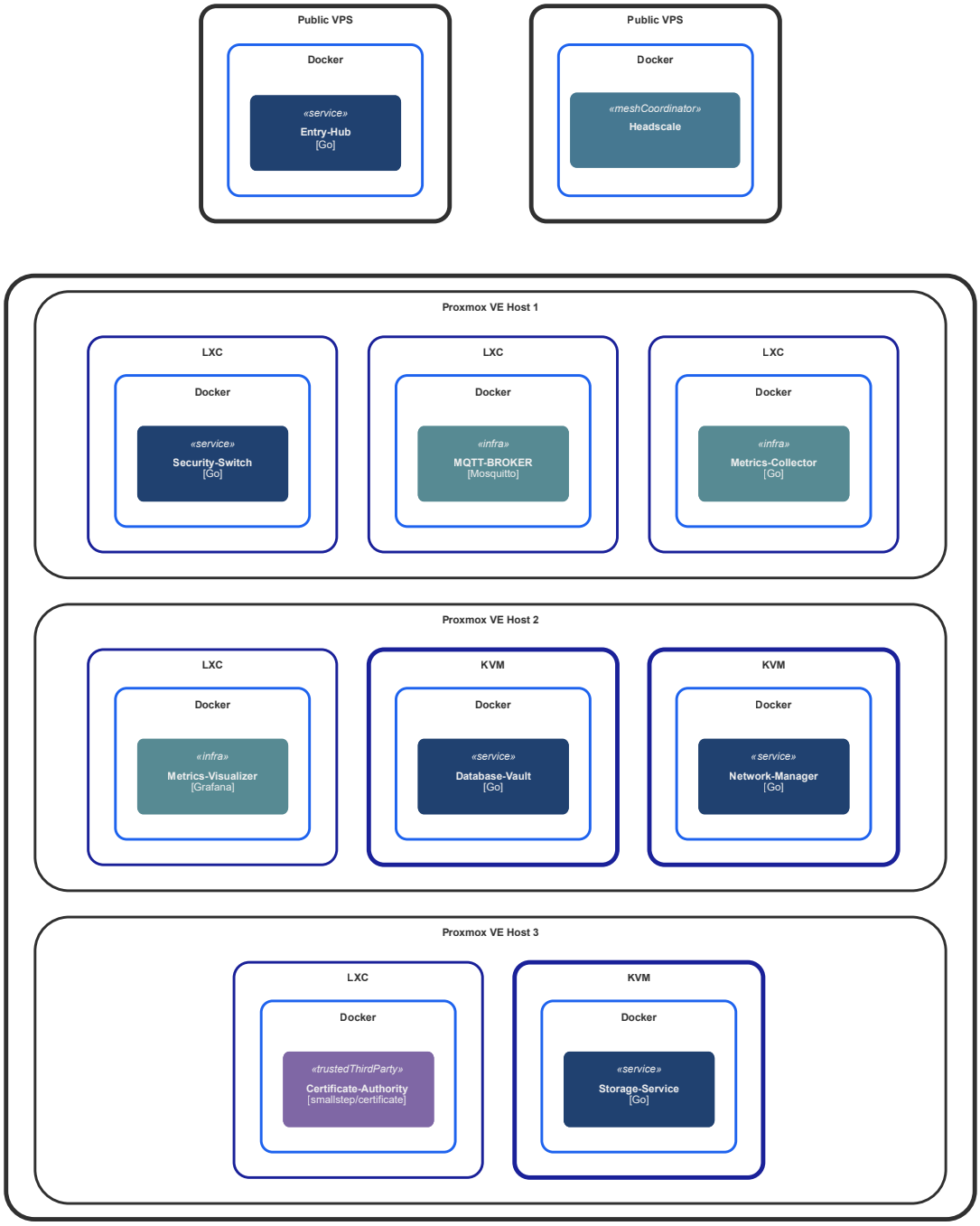


Figure 10.1: Target production topology.

10.5 High availability and resilience: current state and limitations

The service remains available to users only as long as the components that take part in a request's path remain available.

The number of available physical nodes determines how much resilience and fault tolerance RAM-USB can achieve: each added node unlocks one more property.

With a single physical node, `s6-overlay` supervision restarts a crashed process inside a Docker container (Section 10.1), but does not cover the unavailability of the physical node the service runs on. Moreover, the most that can be achieved is local disk redundancy, for example with RAID-10.

A second node unlocks zero-downtime migration of KVMs from one node to another, to balance the load or free one up for maintenance [78]. LXC containers, on the other hand, cannot be live-migrated, only restarted elsewhere [79].

With at least three nodes and storage replicated with Ceph, the system becomes a hyperconverged cluster, able to offer automatic recovery after the failure of a physical node, in addition to the planned relocation of a healthy one.

The nodes, however, must be on the same low-latency network in order to cooperate, since the Proxmox cluster stack requires a latency below 5 ms between all nodes [80] to operate stably.

Even with the best fiber available, two nodes far apart cannot reach such a low latency. Extending the Proxmox backend to multiple sites would require a different mechanism, not yet planned. At present, hyperconvergence and geo-distribution are mutually exclusive. Achieving both at the same time remains an open problem outside the scope of RAM-USB.

11

Critical analysis of security properties

11.1 Attack scenarios and mitigations

The previous chapters describe RAM-USB's defenses from the point of view of those who build them. This section rereads them from the point of view of those who try to compromise them.

11.1.1 Email address enumeration at registration

`POST /api/users` is a public endpoint that responds 201 if the submitted email is available and 409 if it is already registered, without specifying whether the conflict comes from the email or from the SSH key. However, anyone who submits a client-validated password and a freshly generated SSH key, and therefore one that is certainly unique, has already ruled out that the conflict is on the SSH key.

Scheme 11.1 summarizes the procedure, with the following symbols:

- e: the candidate email address;
- p: a client-validated password;
- k: an SSH public key generated by the attacker;
- r: the HTTP status code returned by `POST /api/users`.

1. $k \leftarrow \text{KeyGen}()$
2. $r \leftarrow \text{POST } \texttt{/api/users}(e,\ p,\ k)$
3. $r = 409 \implies e$ already registered

Scheme 11.1: Checking whether an email address is already registered.

Login, on the other hand, hides this information much better, returning the same 401 both for a nonexistent email and for a wrong password, with the same log and the same response time (Section 5.4). That protection was never extended to registration.

The registration endpoint also has a side effect far more serious than enumeration, caused by the fact that the system does not verify that whoever submitted the registration credentials actually owns them.

A malicious actor who registers with an email that does not belong to them permanently prevents registration by its legitimate owner, forcing the owner to use a different email address.

Both problems could be solved by responding to registration requests identically in all cases, for example always with a 202, while the actual outcome would travel out-of-band, in a confirmation email sent to the given address.

This, however, requires rethinking how to deliver to the user the pre-auth key that currently travels inside the 201 registration response.

11.1.2 Administrative access to credentials

Email and password pass, encrypted in transit, through Entry-Hub, Security-Switch, and Database-Vault. Whoever controls even just one of these processes can read them at the point where they are in plaintext in memory.

At rest, however, the password is never recoverable, not even by an administrator with full access to the database. The email, instead, is recoverable by an administrator who holds the master key.

The problem of credentials captured in transit could be mitigated by encrypting the credentials end-to-end between the client and Database-Vault, but in this case the problem would still not be completely solved, and additional complexity would be introduced into the system. The only real solution is to use a zero-knowledge authentication system.

11.1.3 Offline address verification from a database copy

An attacker who holds a stolen copy of the database can immediately check whether a suspected email address is present in the database or not. The attacker only needs to compute the SHA-256 hash of the candidate address and compare it with the hashes stored in the database.

Extracting an arbitrary email from the database, however, requires a very different effort. The attacker cannot decrypt the email without the master key and is therefore forced to attack its SHA-256 hash. The attacker could, however, identify some emails if they appeared in public dictionaries, using a dictionary attack; otherwise, the only viable path remains a brute-force attack.

Scheme 11.2 formalizes the procedure, with the following symbols:

- D: the set of digests present in the stolen database;
- W: a dictionary of plausible addresses;
- e: an email address drawn from W.

1. $\forall e \in W$:
2. $k_h \leftarrow \text{SHA-256}(\text{lower}(e))$
3. $k_h \in D \implies e$ already registered

Scheme 11.2: Search for registered addresses from the stolen database, by dictionary or brute force.

Both of these vulnerabilities can be mitigated by replacing the plain digest with an HMAC-SHA256 keyed with a dedicated pepper, generated with sufficient entropy and distinct from the one used for

password hashing. An attacker who steals the database does not automatically obtain the pepper as well, and therefore cannot recover the email addresses in a reasonable time.

The theft of user credentials, although very serious, does not compromise the backups and does not prevent the user from accessing them. Accessing them requires the SSH private key, which the user keeps locally. Nor are the stolen credentials enough to delete or modify the user's account, because the system does not guarantee users the right to be forgotten.

11.2 Failure isolation

This section analyzes which of the User Requirements remain guaranteed even in the worst case: the full compromise of a service. It also analyzes how to resolve the compromise where possible, provided that it is detected.

Table 11.1: User Requirements verified in this section against the full compromise of a service.

Requirement	Description
RU-04	Be able to access one's own backup only after having authenticated.
RU-05	Have the guarantee that no one, administrators included, can read one's own backups.
RU-06	Have the guarantee that no one, administrators included, can read one's own access credentials.
RU-08	Have the certainty that one's own data is isolated from that of other users.

The remaining properties fall outside this verification: RU-01, RU-02, RU-03, RU-07, and RU-10 are functional or availability requirements that a compromised service easily denies, and therefore do not require a case-by-case verification; RU-09 remains out of scope as stated in Chapter 3.

11.2.1 Entry-Hub

An attacker who gains control of Entry-Hub can read, block, or tamper with every registration and login request. They can also abuse its reachability toward Security-Switch, Certificate-Authority, and the MQTT broker.

Tampering, however, does not affect RU-05, because the Restic repository password originates on the user's device and never passes through Entry-Hub, before, during, or after tampering (Chapter 6). Nor does it affect RU-04, because an already authenticated user reaches Storage-Service directly over the mesh, without ever going through Entry-Hub (Chapter 4). The attacker can, however, block or alter new registration and login requests.

The compromise, if detected, can easily be contained. Network-Manager can revoke a node's membership in the mesh, cutting the compromised Entry-Hub instance off from the network, and while the cause of the compromise is being investigated, a new Entry-Hub instance can be started in a KVM on Proxmox, ideally on a different node (Section 10.3). In this way, registration of new users remains impossible, since the KVM might not have a reachable public address, but user login can take place without problems, since it happens on the handler reachable only from the private mesh network.

11.2.2 Certificate-Authority

An attacker who gains control of Certificate-Authority could sign a valid mTLS certificate for any `organization` (Section 4.4) and thus impersonate any internal service, or refuse to issue updated certificates to services that request a renewal.

Realistically, however, they cannot contact anyone other than the MQTT broker, because according to Headscale's ACL rules, the CA can only respond to calls from internal services, without being able to initiate new communications (Section 4.3).

This does not affect RU-05 or RU-06, since the CA plays no role in backup encryption (Section 6.3) nor in the derivation of the credential keys (Chapter 5). RU-08 also remains guaranteed, as it has no connection with the CA. What breaks down is mutual authentication between services. The services, however, being unable to renew their certificates, can no longer communicate, so once the certificates expire, the entire system comes to a halt.

The failure of authentication between services can easily be detected. When this happens, until the investigation into the compromise is concluded, a new instance of the CA can be restarted in a KVM on Proxmox, ideally on a different node, ensuring greater isolation than that offered by an LXC (Section 10.3).

11.2.3 Database-Vault

An attacker who gains full control of Database-Vault obtains the pepper and the master key, and with the latter decrypts every email stored in the database. However, they do not obtain the password with the same ease. For that, the attacker has two possible paths: the first is to steal the database and attack it offline, at the cost of Argon2id's computation time (Table 5.2). This approach normally requires a great deal of time and considerable computational resources, making it practically infeasible. Without the pepper to protect them, however, weak and very common passwords can easily be cracked with a dictionary attack, even if protected by a slow and expensive hashing algorithm such as Argon2id.

The second path is far more insidious: the attacker can remain passive and observe the calls to Database-Vault. In this way, not only can they read the credentials of new users, but, by analyzing login requests and following them all the way to the database, they can identify which hashes the passwords sent by users correspond to. This requires the malicious actor to remain hidden for the time needed to receive login requests from all the users who registered before the infiltration.

RU-06 therefore no longer holds at all for the email, and is reduced to a time cost for complex and uncommon passwords.

RU-05 nonetheless remains guaranteed: Database-Vault has no access to Storage-Service's filesystem; it only has the ability to ask it to create an account (Chapter 4). RU-08 also remains guaranteed, for the same reason.

In this case, even if the active attack were detected, there would be nothing left to do; one must assume that the database has already been stolen. If instead it were the passive attack that was detected, while the compromise is being investigated, the registration and login services can be shut down completely, and users who are already authenticated can be granted a reachability tag toward Storage-Service that lasts longer than the twelve-hour one they already hold (Chapter 7). In this way,

logged-in users can continue to manage their backups without having to re-authenticate.

In both cases, however, it would be more sensible to shut down the service and declare the incident.

11.2.4 Network-Manager

An attacker who gains control of Network-Manager can grant themselves reachability toward every other service and toward the users (Chapter 7). They can also cancel or extend the access grants toward Storage-Service (Code 7.3). They can therefore block and control all of the system's traffic, without, however, being able to read or decrypt it. RU-04 is therefore compromised. They still cannot access users' data or their credentials (Chapter 5), so RU-05, RU-06, and RU-08 remain valid.

In case of compromise, while the causes are being investigated, Network-Manager can still be shut down and a new instance started in a different KVM, ideally on a different node.

11.2.5 Storage-Service

An attacker who gains control of Storage-Service inherits the root privilege with which the service must run (Section 6.2), and with it reads every user's subdirectory directly from the host filesystem: here RU-08 is violated.

RU-04 is violated as well, because the attacker can prevent any communication with Storage-Service. They can even encrypt, corrupt, or delete the backups of all users, but this is a violation of RU-09, declared out of scope for this project. RU-05 nonetheless remains guaranteed, because the data is encrypted client-side and never arrives in plaintext on Storage-Service (Section 6.3). RU-06 also remains guaranteed, because access credentials do not pass through Storage-Service.

In this case, when the compromise is detected, there is nothing else to do but declare the incident and shut down the service.

12

Conclusions

12.1 Achieved results

Chapter 1 sets the objective of designing, implementing, and validating RAM-USB as a multi-user remote backup service compliant with the requirements of the SRS [9], relying on the zero-trust and defense-in-depth principles to defend the system from external attacks.

The ten user requirements listed in Chapter 3 are, with only two exceptions, fully covered by system requirements implemented and traced all the way to the code and the tests. A user can register by providing only an email, a password, and an SSH public key (RU-01), authenticate in subsequent sessions (RU-02), upload and retrieve client-side encrypted backups from anywhere (RU-03, RU-07), do so only after having authenticated (RU-04), with the guarantee that the content of the backups is unreadable even by an administrator (RU-05), in a space isolated from that of other users (RU-08). An administrator can observe the health and performance of the system without ever touching individual users' data (RU-10).

RU-06 (Have the guarantee that no one, administrators included, can read one's own access credentials) is not fully supported because of the theoretical limitations imposed by the choice of an email and password authentication system, and therefore holds only for the password and only once it is already persisted in the database in hashed form. The only way to prevent an administrator from intercepting credentials in transit, in fact, is not to send them to the server at all, for example by using a zero-knowledge protocol, which is not the subject of this thesis.

RU-09, the guarantee that no one can modify or delete a backup, was declared out of scope from the earliest versions of the SRS, because of the project's tight schedule, and is taken up again in the next section.

The five use cases that run through the system (registration, login, backup, restore, and consulting the metrics) were executed end-to-end on the real Proxmox deployment, with the same results reported in Chapter 9.

12.2 Known limitations

The SRS explicitly accepts four risks as the cost of releasing on schedule [9]. Normally these risks would not be acceptable in a real backup system, and their resolution is planned as future work.

RISK-01 is RU-09 itself. It remains a desirable improvement, not yet designed, implemented, or tested.

RISK-02 concerns the master key and the pepper: they reside in an environment variable, with no backup procedure (DV-F-18) or rotation procedure (DV-F-19). Their loss would entail the irreversible loss of access to all encrypted credentials.

RISK-03 concerns the client: it is currently not containerized with Docker. The service's Go code exists and has been tested, but containerization requires further reasoning and design decisions.

RISK-04 is the circular dependency of Network-Manager's DNS resolution on its own MagicDNS: if Network-Manager published a non-working ACL policy, it could also lose the ability to resolve the Headscale hostname it would need to correct it. In that case, it is up to an administrator to resolve the problem manually.

12.3 Future work

Section 5.2.1 shows that, under uniform choice, the constraint of at least three character categories is worth less than a tenth of a bit of entropy, while one additional character is worth 6.57 bits: under that assumption the composition rule is redundant, because a truly random password already uses several categories by itself. Its real purpose, however, is not to increase entropy but to constrain a human's choice, and in that respect its value cannot be measured without the real distribution of the passwords chosen by users. This design choice therefore currently remains open, pending more in-depth analysis.

Besides DV-F-18 and DV-F-19, already discussed as RISK-02, two items remain to be designed: the per-user storage space limit and the automatic replication of backups on CephFS, both deferred in the SRS as desirable improvements rather than project constraints.

Other future work, not yet designed or included in the SRS, comprises automatic load balancing, protection of Entry-Hub's public endpoint from availability attacks, perhaps by imposing rate limiting on that endpoint, and improved system scalability, perhaps by unlocking geo-distribution of the server.

A

Complete list of requirements

The following tables collect the functional, non-functional, and domain system requirements, organized according to the same structure as the original SRS [9].

Chapter 3 discusses the user requirements in full; they are the only ones not included in the tables below.

A.1 Functional system requirements

A.1.1 Client

Table A.1: Functional requirements of the client.

ID	Requirement
CL-F-01	Must autonomously generate an SSH key pair (public/private) before registration, without ever transmitting the private key to any component of the system.
CL-F-02	On user command, must send to Entry-Hub, during registration (`POST /api/users`), only the SSH public key, together with email and password.
CL-F-03	On user command, must repeat the login (`POST /api/login`) before the expiry of the twelve-hour ACL grant, to maintain continuity of access to Storage-Service.
CL-F-04	Must configure and start Tailscale using the pre-auth key received in the registration response, to join the private mesh network.
CL-F-05	Must resolve Storage-Service through MagicDNS on the mesh network, without relying on static IP addresses.
CL-F-06	On user command, must invoke `restic backup` toward Storage-Service via SFTP on TCP port 2222 (ST-F-03), authenticating with the SSH private key generated in CL-F-01.
CL-F-07	On user command, must invoke `restic restore` toward Storage-Service via SFTP on TCP port 2222 (ST-F-03), using the same authentication method as CL-F-06.

ID	Requirement
CL-F-08	Must handle the HTTP status codes returned by Entry-Hub (400/401/403/500/502/503/504) without exposing internal system details to the end user.
CL-F-09	On user command, must locally validate email, password and, only during registration, the SSH public key, using the same rules enforced by Entry-Hub (EH-F-04 for registration, EH-F-05 for login), before sending `POST /api/users` or `POST /api/login`; if local validation fails, it must not transmit the request.
CL-F-10	Must derive the Entry-Hub login endpoint from a configuration value distinct from the registration one, and must use for login a name covered by the certificate that Entry-Hub presents.
CL-F-11	Must pin the Storage-Service SSH host key: it records the key received in the registration response (ST-F-16) before the first backup, presents it to `ssh` as a known host, and aborts the connection if the presented key differs.

A.1.2 Entry-Hub

Table A.2: Functional requirements of Entry-Hub.

ID	Requirement
EH-F-01	Must expose a public HTTPS health-check endpoint (`GET /api/health`), with certificates signed by the Let's Encrypt public CA.
EH-F-02	Must expose a public HTTPS endpoint for user registration (`POST /api/users`), with certificates signed by the Let's Encrypt public CA.
EH-F-03	Must expose an HTTPS endpoint (`POST /api/login`) for the authentication of registered users, served on a separate listener reachable only from the private mesh network (not from the internet, unlike the listener of EH-F-01/EH-F-02), with the same certificate signed by the Let's Encrypt public CA presented by EH-F-01/EH-F-02, not mTLS; only the network reachability changes.
EH-F-04	`/api/users` must accept JSON and validate: presence of the email, password, and SSH public key fields; payload size within a defined limit; absence of unexpected additional fields; email format (RFC 5322); password length between 8 and 128 characters; password complexity (at least 3 character categories among lowercase, uppercase, digits, symbols); formal correctness of the SSH public key.
EH-F-05	`/api/login` must accept JSON and validate: presence of the email and password fields; payload size within a defined limit; absence of unexpected additional fields; email format (RFC 5322); password length between 8 and 128 characters; password complexity (at least 3 character categories).
EH-F-06	If validation fails, it must: respond with HTTP 400 (Bad Request) without specifying which problem was found; log the problem found without identifying the user; not forward the request to any other internal service.

ID	Requirement
EH-F-07	If validation succeeds, it must: log the validation outcome without identifying the user; forward the request to Security-Switch via mTLS; verify that the certificate comes from a Security-Switch; verify the validity of the X.509 certificate.
EH-F-08	Must forward to the user the response received from Security-Switch.
EH-F-09	Must map internal errors to HTTP status codes 400/401/500/502/503, returning sanitized messages to the client and keeping detailed logs only internally.
EH-F-10	Must publish metrics every minute, and only then, on its own dedicated MQTT topic (`metrics/Entry-Hub`), via mTLS, verifying that: the certificate comes from an MQTT-Broker; the X.509 certificate is valid.
EH-F-11	Metrics must never contain users' personal data, only aggregate statistics.

A.1.3 Security-Switch

Table A.3: Functional requirements of Security-Switch.

ID	Requirement
SS-F-01	Must accept only mTLS connections from clients that satisfy: `organization` field equal to `EntryHub`; valid X.509 certificate; access to the private mesh network.
SS-F-02	Must revalidate the received input, independently of the validation already performed by Entry-Hub.
SS-F-03	If validation fails, it must: respond with HTTP 400 (Bad Request) without specifying which problem was found; log the problem found without identifying the user; not forward the request to any other internal service.
SS-F-04	If validation succeeds, it must: log the validation outcome without identifying the user; forward the request to Database-Vault via mTLS, verifying that: the certificate comes from a Database-Vault; the X.509 certificate is valid.
SS-F-05	After Database-Vault confirms successful authentication, it must request Network-Manager (via mTLS) to grant that user access to Storage-Service for twelve hours.
SS-F-06	Must map errors to HTTP status codes 400/401/403/500/502/504.
SS-F-07	Must publish metrics every minute, and only then, on its own dedicated MQTT topic (`metrics/Security-Switch`), via mTLS, verifying that: the certificate comes from an MQTT-Broker; the X.509 certificate is valid.
SS-F-08	Metrics must never contain users' personal data, only aggregate statistics.
SS-F-09	After Database-Vault confirms successful registration, it must request Network-Manager (via mTLS) to create a dedicated Headscale user and generate a pre-auth key for the new account, and then include that key in the response to Entry-Hub.

A.1.4 Database-Vault

Table A.4: Functional requirements of Database-Vault.

ID	Requirement
DV-F-01	Must accept only mTLS connections from clients that satisfy: `organization` field equal to `SecuritySwitch`; valid certificate; access to the private mesh network.
DV-F-02	Must revalidate the received input, independently of the validation already performed by Security-Switch.
DV-F-03	Must compute the SHA-256 hash of the email for indexing and as primary key, without ever logging the email in plaintext.
DV-F-04	Must encrypt the user's email: derive a per-record encryption key from the master key with HKDF-SHA256 and a 16-byte random salt, then encrypt the email with AES-256-GCM using that derived key and a 12-byte random nonce.
DV-F-05	The master key should come from a configurable source, with length validation (32 bytes).
DV-F-06	Must keep a pepper as an environment variable.
DV-F-07	Must compute the password hash with Argon2id: memory 47104 KiB (46 MiB), 2 iterations, parallelism 1, 32-byte output, using a per-record random salt and the pepper (DV-F-06).
DV-F-08	Must save the user record in an atomic transaction.
DV-F-09	Must request Storage-Service to create the unique POSIX user on the server, with username `user<xxxxxx>` (`xxxxxx`: 6 random characters from a base-36 alphabet), and wait for its response.
DV-F-10	If the creation of the POSIX user fails, it must delete the user from the database and inform Security-Switch that the registration failed.
DV-F-11	After creating the user record and the POSIX user, it must inform Security-Switch that the user has been registered.
DV-F-12	Must reject (HTTP 409) registrations with an email or an SSH key that already exist, without providing details about the error.
DV-F-13	During login, it must retrieve the salt associated with the email through the SHA-256 hash of the email (DV-F-03).
DV-F-14	Must recompute Argon2id on the received password using the retrieved salt and the pepper, comparing the result with the stored hash.
DV-F-15	Must respond with the same HTTP 401 code both for a nonexistent email and for a wrong password, without distinguishing the two cases either in the response or in the log.
DV-F-16	Must publish metrics every minute, and only then, on its own dedicated MQTT topic (`metrics/Database-Vault`), via mTLS, verifying that: the certificate comes from an MQTT-Broker; the X.509 certificate is valid.
DV-F-17	Metrics must never contain users' personal data, only aggregate statistics.
DV-F-18	A backup procedure for the master key should exist.
DV-F-19	A rotation procedure for the master key should exist.

ID	Requirement
DV-F-20	If validation fails, it must: respond with HTTP 400 (Bad Request) without specifying which problem was found; log the problem found without identifying the user; not forward the request to any other internal service.

A.1.5 Storage-Service

Table A.5: Functional requirements of Storage-Service.

ID	Requirement
ST-F-01	Must accept mTLS connections only from clients that satisfy: `organization` field equal to `DatabaseVault`; valid certificate; access to the private mesh network.
ST-F-02	Must provide upload/download of files already encrypted client-side, without ever processing plaintext content.
ST-F-03	Access must occur exclusively via SFTP on TCP port 2222, authenticated with the SSH public key registered by the user.
ST-F-04	Must explicitly forbid any form of SSH connection other than SFTP.
ST-F-05	Each user must have an isolated storage space, not accessible by other users.
ST-F-06	On request from Database-Vault via mTLS, it must create on the system a POSIX user with username `user<xxxxxx>` (`xxxxxx`: 6 random characters from a base-36 alphabet).
ST-F-07	Must guarantee that the POSIX user can never leave its own directory.
ST-F-08	The created POSIX account must have neither a traditional home directory nor an interactive shell; the only writable space is the dedicated subdirectory inside the chroot.
ST-F-09	The Storage-Service sshd configuration must have `PasswordAuthentication no` and `PermitRootLogin no`, regardless of the fact that the created accounts have no password set.
ST-F-10	Following the creation, successful or failed, of the POSIX user, it must report the outcome to Database-Vault.
ST-F-11	At every SFTP connection attempt by the user, it must retrieve the user's current public key from Database-Vault through `AuthorizedKeysCommand`.
ST-F-12	Must publish metrics every minute, and only then, on its own dedicated MQTT topic (`metrics/Storage-Service`), via mTLS, verifying that: the certificate comes from an MQTT-Broker; the X.509 certificate is valid.
ST-F-13	Metrics must never contain users' personal data, only aggregate statistics.
ST-F-14	Should enforce per-user quota limits.
ST-F-15	Should guarantee: automatic data replication; fault tolerance for at least one node; data consistency; the possibility of expansion by adding new nodes without interrupting the service.

ID	Requirement
ST-F-16	Must return its own SSH host public key in the response to the POSIX user creation request coming from Database-Vault (ST-F-10), so that it can be forwarded to the client and pinned by it (CL-F-11).

A.1.6 Network-Manager

Table A.6: Functional requirements of Network-Manager.

ID	Requirement
NM-F-01	Must guarantee that only Entry-Hub can contact Security-Switch.
NM-F-02	Must guarantee that only Security-Switch and Storage-Service can contact Database-Vault.
NM-F-03	Must guarantee that only Security-Switch can contact Network-Manager.
NM-F-04	Must guarantee that all internal components of the network, except users, can contact Certificate-Authority on the mesh network.
NM-F-05	Must guarantee that only authenticated users can see and contact Storage-Service.
NM-F-06	Must guarantee that registered but unauthenticated users can see and contact only Entry-Hub.
NM-F-07	Must guarantee that registered and authenticated users can see and contact only Entry-Hub and Storage-Service.
NM-F-08	On request from Security-Switch, following a successful registration, it must create a dedicated Headscale user and generate a short-lived pre-auth key for the new account.
NM-F-09	After a successful login, on request from Security-Switch, it must assign to the user's node the ACL tag that enables reachability toward Storage-Service, recording an expiry twelve hours from that moment.
NM-F-10	Must periodically check the recorded expiries and remove the ACL tag from expired nodes, automatically and without manual intervention.
NM-F-11	The expiry of each grant must be persisted, not kept only in memory, so as not to lose state if Network-Manager restarts.
NM-F-12	Pre-auth key creation and ACL tag management must be restricted specifically to Network-Manager, verified via mutual TLS (`organization` field equal to `NetworkManager`).
NM-F-13	The pre-auth key serves solely to register the node as a member of the mesh; on its own, it does not grant reachability toward Storage-Service.
NM-F-14	The Headscale coordination endpoint is deliberately reachable from the public network (ideally hosted on its own VPS with a public address).
NM-F-15	Must configure MagicDNS with a dedicated base domain, so that Storage-Service is resolvable by all mesh nodes through a stable name rather than an IP.

ID	Requirement
NM-F-16	The Network-Manager mesh node must accept the DNS configuration distributed by Headscale (MagicDNS), so that its own outbound calls toward Certificate-Authority and MQTT-Broker resolve those hostnames on the mesh, rather than falling back to a network path with no equivalent in production.
NM-F-17	Must publish metrics every minute, and only then, on its own dedicated MQTT topic (`metrics/Network-Manager`), via mTLS, verifying that: the certificate comes from an MQTT-Broker; the X.509 certificate is valid.
NM-F-18	Metrics must never contain users' personal data, only aggregate statistics.
NM-F-19	Must configure MagicDNS with a record that resolves the public name of Entry-Hub to its mesh address, so that a network member contacting the login listener (EH-F-03) uses the name covered by the Entry-Hub certificate.
NM-F-20	Must guarantee that only the services that publish metrics (Entry-Hub, Security-Switch, Database-Vault, Storage-Service, Network-Manager, Certificate-Authority) and Metrics-Collector can contact MQTT-Broker.
NM-F-21	Must guarantee that only Grafana can contact Metrics-Collector.

A.1.7 Certificate-Authority

Table A.7: Functional requirements of Certificate-Authority.

ID	Requirement
CA-F-01	Must guarantee that the components presenting certificates for mTLS really are who they claim to be.
CA-F-02	Must guarantee the issuance, rotation, revocation, and verification of mTLS certificates.
CA-F-03	Must publish metrics every minute, and only then, on its own dedicated MQTT topic (`metrics/Certificate-Authority`), via mTLS, verifying that: the certificate comes from an MQTT-Broker; the X.509 certificate is valid.
CA-F-04	Must accept a single-use bootstrap token, distributed out-of-band to each service, for issuing the initial certificate; subsequent renewals must occur through the current mTLS certificate, not through the token.

A.1.8 Monitoring system

Table A.8: Functional requirements of the monitoring system (MQTT-Broker / Metrics-Collector / TimescaleDB / Grafana).

ID	Requirement
MT-F-01	Metrics-Collector can read only `metrics/*`.

ID	Requirement
MT-F-02	Metrics-Collector must discard metrics whose `service` field does not match the topic they come from.
MT-F-03	Metrics must be stored as a TimescaleDB hypertable, with automatic retention of 30 days and compression after 7 days.
MT-F-04	Grafana dashboards must exist for response time, throughput, and active connections.

A.1.9 Network infrastructure and PKI

Table A.9: Functional requirements of network and PKI.

ID	Requirement
PKI-F-01	Every service must mutually authenticate with X.509 certificates issued by a valid CA.
PKI-F-02	Every service must verify the `organization` field of the certificate, not only its validity.
PKI-F-03	A certificate rotation and revocation procedure should exist.
NET-F-01	Communication between services must occur on the private network; the public endpoints of Entry-Hub and the Headscale coordination endpoint of Network-Manager (NM-F-14) are the only surfaces deliberately exposed publicly by the system.
NET-F-02	The TLS used must be version 1.3.

A.2 Non-functional requirements

A.2.1 Product

Table A.10: Product non-functional requirements.

ID	Requirement
RNF-SEC-01	No server-side component ever accesses the content of backup files in plaintext, since encryption happens client-side before transmission. This does not extend to access credentials: email and password transit, encrypted (TLS/mTLS), through Entry-Hub, Security-Switch, and Database-Vault for validation and hashing, while never being persisted or logged in plaintext (cf. DV-F-03, RD-01).
RNF-SEC-02	Zero-trust: no service must implicitly trust the data received from another, even if authenticated via mTLS.
RNF-SEC-03	Defense-in-depth: every layer revalidates input independently.
RNF-SEC-04	All communication between services must use mTLS, without exceptions.
RNF-REL-01	The system must tolerate the isolated compromise of a single component, without it propagating to the others.

ID	Requirement
RNF-PERF-01	The latency of HTTP requests must be tracked (p50/p95/p99).
RNF-USA-01	Error messages must be understandable and correctly categorized by HTTP status code.
RNF-MAINT-01	Every service must be able to be isolated, re-certified, and restarted individually without impacting the others.

A.2.2 Organizational

Table A.11: Organizational non-functional requirements.

ID	Requirement
RNF-ORG-01	Implementation language: Go.
RNF-ORG-03	MIT open-source license.
RNF-ORG-04	Deployment platform: Proxmox VE (KVM for Storage-Service, Database-Vault, Network-Manager; LXC for the other services).
RNF-ORG-05	Development and operation guaranteed on macOS and Linux (with Docker).

A.2.3 External

Table A.12: External non-functional requirements.

ID	Requirement
RNF-EXT-01	Since the system processes personal data (the email), it should comply with the applicable privacy regulations (e.g., GDPR). Currently out of scope.

A.3 Domain requirements

Table A.13: Domain requirements (RD).

ID	Requirement
RD-01	Any new component introduced in the future must not create a path along which sensitive data in plaintext traverses or is logged by a component other than the client or the component strictly necessary for its encryption/decryption.
RD-02	Derived keys (via HKDF) must never be persisted: any new key storage requirement must be evaluated against this constraint before being accepted.
RD-03	Argon2id and AES-256-GCM are non-negotiable technological constraints.
RD-04	The *fail-secure* principle applies to every component: in case of uncertainty about the validity of a request, the default behavior is to deny access.

Bibliography

[1] Francesco Verrengia. RAM-USB: creation and ownership of each user's chroot root and `data` directory.
`https://github.com/Verryx-02/RAM-USB/blob/3f20762d732f412fc1e724c41658a3d493a781a5/services/storage-service/internal/posixuser/creator.go#L157-L168`, 2026.

[2] Francesco Verrengia. RAM-USB: permission bits of each user's chroot root and `data` directory.
`https://github.com/Verryx-02/RAM-USB/blob/3f20762d732f412fc1e724c41658a3d493a781a5/services/storage-service/internal/posixuser/dirmaker.go#L36-L50`, 2026.

[3] David A. Patterson, Garth Gibson, and Randy H. Katz. A case for redundant arrays of inexpensive disks (RAID). In *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*, pages 109–116, New York, NY, USA, 1988. ACM.
`https://doi.org/10.1145/50202.50214`.

[4] Riccardo Gottardi. GitHub profile.
`https://github.com/Riccardo-Gottardi`, 2026.

[5] Barry W. Boehm. A spiral model of software development and enhancement. *Computer*, 21(5):61–72, 1988.
Paywalled IEEE: `https://doi.org/10.1109/2.59`
Freely accessible: `https://www.cse.msu.edu/~cse435/Homework/HW3/boehm.pdf`.

[6] Francesco Verrengia. RAM-USB contributing guidelines.
`https://github.com/Verryx-02/RAM-USB/blob/main/CONTRIBUTING.md`, 2026.

[7] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. NIST special publication 800-207: Zero Trust Architecture.
`https://doi.org/10.6028/NIST.SP.800-207`, 2020.

[8] National Institute of Standards and Technology. NIST special publication 800-53 revision 5: Security and privacy controls for information systems and organizations.
`https://doi.org/10.6028/NIST.SP.800-53r5`, 2020.

[9] Francesco Verrengia. RAM-USB software requirements specification.
`https://github.com/Verryx-02/RAM-USB/blob/main/docs/Software_Requirements_Specification.md`, 2026.

[10] Francesco Verrengia. RAM-USB software requirements specification, v1.0: original wording of NM-F-12.
`https://github.com/Verryx-02/RAM-USB/blob/3ef723e07a0292337fc48a1cd2e8c952f2f2e2ee/docs/Software_Requirements_Specification.md?plain=1#L260`, 2026.

[11] Francesco Verrengia. RAM-USB: correct NM-F-12 wording and document how headscale's CLI satisfies it.
`https://github.com/Verryx-02/RAM-USB/commit/975be1f467873628532a807e852f086dba29a9ef`, 2026.

[12] Francesco Verrengia. RAM-USB: rework NM-F-12/NM-F-14/NET-F-01 for a standalone public headscale and retire EH-F-12.
https://github.com/Verryx-02/RAM-USB/commit/f2d17f41668ae2c978c849ac308ac4fa52f7b409, 2026.

[13] Francesco Verrengia. RAM-USB contributing guidelines: Commit structure (section 2.2).
https://github.com/Verryx-02/RAM-USB/blob/7984a140e0ce7f6b3ea89470801c0d26ecbd4aa1/CONTRIBUTING.md?plain=1#L33-L101, 2026.

[14] Google. distroless: Language focused docker images, minus the operating system.
https://github.com/GoogleContainerTools/distroless, 2026.

[15] Murugiah Souppaya, John Morello, and Karen Scarfone. Application container security guide.
https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-190.pdf, 2017. NIST Special Publication 800-190.

[16] Michael Wager. Evaluating the security of containers through reduction of potentially vulnerable components.
https://www.mwager.de/assets/component_reduction_paper.pdf, 2024. Hochschule Augsburg.

[17] Josh Aas, Richard Barnes, Benton Case, Zakir Durumeric, Peter Eckersley, Alan Flores-López, J. Alex Halderman, Jacob Hoffman-Andrews, James Kasten, Eric Rescorla, Seth Schoen, and Brad Warren. Let's Encrypt: An automated certificate authority to encrypt the entire web. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*, pages 2473–2487, New York, NY, USA, 2019. ACM. Paywalled (ACM Digital Library): https://doi.org/10.1145/3319535.3363192 Freely accessible (hosted by ISRG, the organization behind Let's Encrypt): https://www.isrg.org/documents/letsencryptCCS2019.pdf.

[18] Francesco Verrengia. RAM-USB: decodeJSON and ValidateRegister.
https://github.com/Verryx-02/RAM-USB/blob/7c1e991f953a4c686e09f8142d0b3c1d9d64fc5f/pkg/validation/validation.go#L125-L173, 2026.

[19] Francesco Verrengia. RAM-USB: shared internal libraries.
https://github.com/Verryx-02/RAM-USB/tree/main/pkg, 2026.

[20] restic. restic: Fast, secure, efficient backup program.
https://github.com/restic/restic, 2026.

[21] Tatu Ylönen and Sami Lehtinen. SSH file transfer protocol.
https://datatracker.ietf.org/doc/html/draft-ietf-secsh-filexfer-02, 2001. the revision implemented by OpenSSH, never published as an RFC.

[22] OWASP. Fail securely.
https://owasp.org/www-community/Fail_securely, 2026.

[23] Smallstep. certificates: A private certificate authority (x.509 & ssh) & acme server for secure automated certificate management.
https://github.com/smallstep/certificates, 2026.

[24] Juan Font. headscale: An open source, self-hosted implementation of the tailscale control server.
https://github.com/juanfont/headscale, 2026.

[25] Timescale. TimescaleDB: A time-series database for high-performance real-time analytics.
https://github.com/timescale/timescaledb, 2026.

[26] Grafana Labs. Grafana: The open and composable observability and data visualization platform. https://github.com/grafana/grafana, 2026.

[27] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[28] Peter W. Resnick. Internet message format. https://doi.org/10.17487/RFC5322, 2008. RFC 5322.

[29] Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). https://doi.org/10.17487/RFC5869, 2010. RFC 5869.

[30] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. https://doi.org/10.6028/NIST.SP.800-38D, 2007. NIST Special Publication 800-38D.

[31] Francesco Verrengia. RAM-USB: per-record key derivation with HKDF-SHA256. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/encryption/encryption.go#L88-L109, 2026.

[32] Alex Biryukov, Daniel Dinu, Dmitry Khovratovich, and Simon Josefsson. Argon2 memory-hard function for password hashing and proof-of-work applications. https://doi.org/10.17487/RFC9106, 2021. RFC 9106.

[33] OWASP. Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2026.

[34] Francesco Verrengia. RAM-USB: saving the user record in an atomic transaction. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/storage/storage.go#L111-L146, 2026.

[35] Francesco Verrengia. RAM-USB: compensating rollback after a storage-service failure. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/registration/registration.go#L157-L165, 2026.

[36] Francesco Verrengia. RAM-USB: single login outcome for nonexistent email and wrong password. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/login/login.go#L160-L185, 2026.

[37] Francesco Verrengia. RAM-USB: dummy verification that equalizes login response times. https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/password/hash.go#L178-L193, 2026.

[38] Francesco Verrengia. RAM-USB: generation and local storage of the Restic repository password. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/user-client/internal/reposecret/reposecret.go#L51-L77, 2026.

[39] restic. restic documentation: references and repository design. https://restic.readthedocs.io/en/stable/100_references.html, 2026.

[40] OpenBSD Project. sshd_config(5): Openssh daemon configuration file. https://man.openbsd.org/sshd_config.5, 2026.

[41] Francesco Verrengia. RAM-USB: Storage-service hardened `sshd_config`. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/deployments/docker/storage-service/sshd_config, 2026.

[42] Francesco Verrengia. RAM-USB: fail-secure exit paths of the `AuthorizedKeysCommand` binary. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/storage-service/cmd/authorized-keys-command/main.go#L97-L132, 2026.

[43] Francesco Verrengia. RAM-USB: Restic repository URL and SFTP transport command. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/user-client/internal/restic/restic.go#L74-L90, 2026.

[44] Francesco Verrengia. RAM-USB: mesh reachability rules published to Headscale. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/policy.go#L198-L336, 2026.

[45] Francesco Verrengia. RAM-USB: mesh identity and pre-auth key creation at registration. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/client.go#L198-L217, 2026.

[46] Francesco Verrengia. RAM-USB: fixed twelve-hour grant duration and assignment of the Storage-Service access tag after login. Grant duration: https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/client.go#L111-L116
Access tag assignment: https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/client.go#L251-L275, 2026.

[47] Francesco Verrengia. RAM-USB: SQLite schema of the permanent email-to-pre-auth-key-ID mapping. https://github.com/Verryx-02/RAM-USB/blob/cd4b9fcaf3b1f503d96040287d4849c8cc17f2ac/services/network-manager/internal/grants/meshusers.go#L20-L25, 2026.

[48] Francesco Verrengia. RAM-USB: SQLite schema of the persisted access-grants table. https://github.com/Verryx-02/RAM-USB/blob/cd4b9fcaf3b1f503d96040287d4849c8cc17f2ac/services/network-manager/internal/grants/store.go#L87-L94, 2026.

[49] Francesco Verrengia. RAM-USB: periodic revocation of expired access grants. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/grants/sweep.go#L54-L88, 2026.

[50] Headscale. Headscale FAQ: can I use headscale and tailscale on the same machine? https://headscale.net/stable/about/faq/, 2026.

[51] Francesco Verrengia. RAM-USB: reverse proxy separating the administrative path, with mTLS, from the coordination path, without checks. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/deployments/docker/headscale/nginx.conf#L110-L139, 2026.

[52] ICANN. Available pool of unallocated IPv4 internet addresses now completely emptied. https://itp.cdn.icann.org/en/files/announcements/release-03feb11-en.pdf, 2011. Press release, February 3, 2011.

[53] Jason Weil, Victor Kuarsingh, Chris Donley, Christopher Liljenstolpe, and Marla Azinger. IANA-reserved IPv4 prefix for shared address space. https://doi.org/10.17487/RFC6598, 2012. RFC 6598, BCP 153.

[54] Simon Perreault, Ikuhei Yamagata, Shin Miyakawa, Akira Nakagawa, and Hiroyuki Ashida. Common requirements for carrier-grade NATs (CGNs).
https://doi.org/10.17487/RFC6888, 2013. RFC 6888, BCP 127.

[55] Mat Ford, Mohamed Boucadair, Alain Durand, Pierre Levis, and Phil Roberts. Issues with IP address sharing.
https://doi.org/10.17487/RFC6269, 2011. RFC 6269.

[56] Eclipse Foundation. Eclipse Mosquitto: an open source MQTT broker.
https://github.com/eclipse-mosquitto/mosquitto, 2026.

[57] Francesco Verrengia. RAM-USB: shape of the aggregated metrics payload.
https://github.com/Verryx-02/RAM-USB/blob/dd0302834b0222ec9ef1899a01735d16725c5137/pkg/metrics/payload.go#L50-L57, 2026.

[58] Andrew Banks and Rahul Gupta. MQTT version 3.1.1 plus errata 01.
https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/errata01/os/mqtt-v3.1.1-errata01-os-complete.html, 2015.

[59] Francesco Verrengia. RAM-USB: hypertable, retention, and compression policy for the metrics table.
https://github.com/Verryx-02/RAM-USB/blob/17dffbd13729426591f7052f8e2b782d41332d67/services/metrics-collector/migrations/000001_create_metrics_table.up.sql#L56-L76, 2026.

[60] Francesco Verrengia. RAM-USB: test that nonexistent email and wrong password produce the same outcome.
https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/services/database-vault/internal/login/login_test.go#L88-L129, 2026.

[61] golangci-lint. `golangci-lint`: aggregated static analyzer for Go.
https://golangci-lint.run/, 2026.

[62] securego. `gosec`: static security analyzer for Go.
https://github.com/securego/gosec, 2026.

[63] timakin. `bodyclose`: detection of unclosed HTTP response bodies.
https://github.com/timakin/bodyclose, 2026.

[64] ryanrolds. `sqlclosecheck`: detection of unclosed SQL rows and statements.
https://github.com/ryanrolds/sqlclosecheck, 2026.

[65] kkHAIKE. `contextcheck`: verification of context.Context propagation.
https://github.com/kkHAIKE/contextcheck, 2026.

[66] polyfloyd. `errorlint`: verification of Go error-wrapping conventions.
https://codeberg.org/polyfloyd/go-errorlint, 2026.

[67] Go Team. `govulncheck`: analysis of vulnerabilities reachable from the code.
https://pkg.go.dev/golang.org/x/vuln/cmd/govulncheck, 2026.

[68] Gitleaks. `gitleaks`: detection of secrets in repositories.
https://github.com/gitleaks/gitleaks, 2026.

[69] Aqua Security. `trivy`: vulnerability scanner for images and filesystems.
https://github.com/aquasecurity/trivy, 2026.

[70] Francesco Verrengia. RAM-USB known issues (scratch cache).
https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/docs/Known_Issues.md, 2026.

[71] just-containers. s6-overlay: process supervisor for Docker containers, built on s6.
https://github.com/just-containers/s6-overlay, 2026.

[72] Francesco Verrengia. RAM-USB: Storage-service's s6-overlay service directories.
https://github.com/Verryx-02/RAM-USB/tree/b7b2914230f844745df007057315c19f57d1a576/deployments/docker/storage-service/rootfs/etc/s6-overlay/s6-rc.d, 2026.

[73] Francesco Verrengia. RAM-USB: Entry-hub's distroless runtime stage.
https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/docker/entry-hub/Dockerfile#L104-L111, 2026.

[74] Proxmox Server Solutions GmbH. Proxmox VE: open-source server virtualization (KVM and LXC).
https://www.proxmox.com/en/proxmox-virtual-environment/overview, 2026.

[75] Docker Inc. What is docker?
https://docs.docker.com/get-started/docker-overview/, 2026.

[76] Francesco Verrengia. RAM-USB: Storage-service Proxmox deployment notes.
https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/proxmox/storage-service.md, 2026.

[77] Francesco Verrengia. RAM-USB: one Docker Compose file per service.
https://github.com/Verryx-02/RAM-USB/tree/b7b2914230f844745df007057315c19f57d1a576/deployments/compose, 2026.

[78] Proxmox Server Solutions GmbH. qm: live migration of KVM virtual machines.
https://git.proxmox.com/?p=pve-docs.git;a=blob_plain;f=qm.adoc;hb=HEAD, 2026.

[79] Proxmox Server Solutions GmbH. pct: migration of LXC containers, no live migration available.
https://git.proxmox.com/?p=pve-docs.git;a=blob_plain;f=pct.adoc;hb=HEAD, 2026.

[80] Proxmox Server Solutions GmbH. Cluster manager: sub-5ms latency requirement between cluster nodes.
https://pve.proxmox.com/pve-docs/chapter-pvecm.html, 2026.